CMS · NANOAOD · TRIGGER OBJECTS · DETAILED WALKTHROUGH

Why the unseeded diphoton leg has no NanoAOD filter bits, from the HLT menu to the fix

Thirty-one slides with the real code at each step: the HLT path and its filters, the trigger summary, PAT and MiniAOD, the NanoAOD table producer and its selection string, the scope across every NanoAOD version and every menu since 2018, a scan of the whole trigger-object table, and the one-line fix with its validation.

Santanu Mahata · 4 November 2026

Sources: cms-sw/cmssw master ed35654b1, CMSSW_13_3_1_patch1 (NanoAOD v13), release branches 12_4_X to 16_1_X, HLT menus 2018 to 2026

`TrigObj_filterBits · bits 16, 17 (v15) · 14, 15 (v12 to v14)`

The problem in one page

3

photon filter bits documented but impossible to set (16, 17, 19 in v15; 14, 15, 17 in v12 to v14)

4+

NanoAOD versions affected: v12, v13, v14, v15 and everything after, including today's master

19

HLT menu files checked, 2018 to 2026: same filter names, same collections, same chain everywhere

1

line to change in triggerObjects_cff.py, mirroring what the Electron selection has done since 2023

Problem

The Photon entry of the NanoAOD trigger-object table only accepts HLT objects from the collection hltEgammaCandidates. The sub-leading leg of HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90 and _Mass95 is built from hltEgammaCandidatesUnseeded. The collection test is applied before the filter bits are computed, so every object carrying the unseeded last-filter labels is dropped and its bits can never appear. The sub-leading photon of the main H to gamma gamma triggers cannot be trigger-matched from NanoAOD. Confirmed on twelve NanoAOD files from v9 to v15: not a single photon object with an unseeded bit.

Solution

Accept both e/gamma collections in the Photon selection string, exactly as the Electron selection does. Because objects are kept only when they set at least one photon bit, and the only photon-bit filters on the unseeded collection are the three diphoton ones, the output changes only in events that fired those triggers. Bit numbers and documentation stay the same. NANO re-made with the fix sets the unseeded bits in all 10453 Mass90 events of the test file and changes nothing else. The fix branch is on the fork; no pull request is open yet.

What a NanoAOD trigger object is, and how analyses use the filter bits

branch	meaning
TrigObj_id	object type: 11 electron, 22 photon, 13 muon, 15 tau, 1 jet, 6 fat jet, 2 MET, 3 HT, 4 MHT
TrigObj_pt, eta, phi	the HLT object's kinematics (for e/gamma: the supercluster)
TrigObj_l1pt, l1iso, l1charge	the matched L1 seed within a cone (type -98 for e/gamma)
TrigObj_l2pt	the matched L2 object where configured (muons, jets, MET)
TrigObj_filterBits	one bit per HLT filter the object passed, from a per-type list in triggerObjects_cff.py; 32-bit up to 15_0_1, 64-bit from 15_0_2

```
# sketch of a typical NanoAOD trigger match (awkward)
to   = events.TrigObj
ph   = to[to.id == 22]
leg2 = ph[(ph.filterBits >> 17) & 1 == 1]
# unseeded R9 leg: always empty today
dr   = photons.metric_table(leg2)
ok   = ak.any(dr < 0.1, axis=2)
```

A leg-by-leg efficiency, a trigger-matched tag-and-probe, or a HiggsDNA-style matching of both photons needs the bit of the last filter of each leg. For the sub-leading leg of the diphoton triggers that bit is never set, so the match always fails.

The table is produced by TriggerObjectTableProducer from the unpacked MiniAOD trigger objects. Each object type has its own selection string; an object is written only if it passes that string and sets at least one bit of that type.

Two e/gamma candidate collections at HLT: L1-seeded and unseeded

hltEgammaCandidates (seeded)

Superclusters built only in regions around L1 e/gamma candidates. Used by the leading leg (Et > 30 GeV, L1 match required) and by nearly every single-photon and single-electron path.

```
fragment.hltEgammaCandidates = cms.EDProducer(
  "EgammaHLTRecoEcalCandidateProducers",
  scHybridBarrelProducer = cms.InputTag(
    'hltParticleFlowSuperClusterECALL1Seeded',
    'hltParticleFlowSuperClusterECALBarrel' ),
  scIslandEndcapProducer = cms.InputTag(
    'hltParticleFlowSuperClusterECALL1Seeded',
    'hltParticleFlowSuperClusterECALEndcapWithPreshower'
),
  recoEcalCandidateCollection = cms.string( "" ) )
```

hltEgammaCandidatesUnseeded

Superclusters built over the whole ECAL, with no L1 requirement. Used for the sub-leading object of double-object paths (Et > 22 GeV here), which may have no L1 seed of its own.

```
fragment.hltEgammaCandidatesUnseeded = cms.EDProducer(
  "EgammaHLTRecoEcalCandidateProducers",
  scHybridBarrelProducer = cms.InputTag(
    'hltParticleFlowSuperClusterECALUnseeded',
    'hltParticleFlowSuperClusterECALBarrel' ),
  scIslandEndcapProducer = cms.InputTag(
    'hltParticleFlowSuperClusterECALUnseeded',
    'hltParticleFlowSuperClusterECALEndcapWithPreshower'
),
  recoEcalCandidateCollection = cms.string( "" ) )
```

Two producers, two RecoEcalCandidate collections, two distinct trigger objects in the event even when they describe the same photon. The trigger summary, PAT and MiniAOD keep the collection name with each object. NanoAOD decides per object type which collection names it accepts, and that decision is where the photon table goes wrong.

HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90: one path, two legs, four last filters

SEEDED LEG · hltEgammaCandidates · L1-seeded · Et > 30 GeV

L1 match, then Et > 30 GeV → R9 > 0.5 (EB) / 0.8 (EE), then H/E → either R9 > 0.85 / 0.90, last filter: **bit 14**; or cluster shape + ECAL iso, last filter: **bit 15** → leg combiner: (R9 or iso) and H/E

UNSEEDED LEG · hltEgammaCandidatesUnseeded · no L1 seed · Et > 22 GeV

HLT1Photon: Et > 22 GeV, |eta| < 2.55, saved as type 92 → R9 > 0.5 (EB) / 0.8 (EE), then H/E → either R9 > 0.85 / 0.90, last filter: **bit 17 never set**; or cluster shape + ECAL + track iso, last filter: **bit 16 never set** → (no combiner; the ignore() filters feed the pair filter directly)

↓ Pair filter hltDiEG22R9Id85b90eORIso60CaloId15b35eANDHE12R9Id50b80eMass90CombMassLastFilter (HLTEgammaAllCombMassFilter): m(γγ) > 90 GeV, Mass95 path: > 95 GeV. No NanoAOD bit.

Bit numbers are those of NanoAOD v15 and master; in v12 to v14 the same four last filters are bits 12, 13 (seeded) and 15, 14 (unseeded). Mass95 shares the whole sequence.

The path and its sequence as defined in the NanoAOD v13 release menu

```
# CMSSW_13_3_1_patch1, HLTrigger/Configuration/python/HLT_FULL_cff.py, line 119468 (path) and 118801 (sequence)
fragment.HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90_v19 = cms.Path(
    fragment.HLTBeginSequence + fragment.hltL1sSingleAndDoubleEGor
  + fragment.hltPreDiphoton3022R9IdORIsoCaloIdANDHER9IdMass90
  + fragment.HLTDiphoton30L22R9Id85b90eORIso60CaloId15b35eANDHE12R9Id50b80eSequence
  + fragment.hltDiEG22R9Id85b90eORIso60CaloId15b35eANDHE12R9Id50b80eMass90CombMassLastFilter
  + fragment.HLTEndSequence )
```

```
# the sequence, e/gamma modules only; cms.ignore(f) = f is run and saves its objects but cannot stop the path
hltEgammaCandidates > hltEGL1SingleAndDoubleEGOrFilter > hltEG30L1SingleAndDoubleEGOrEtFilter
  > hltEG30LR9Id50b80eR9IdFilter > hltEG30LHE12R9Id50b80eHEFilter
  > ignore(hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter)            # bit 14
  > ignore(hltEG30LCaloId15b35eHE12R9Id50b80eClusterShapeFilter)
  > ignore(hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter)        # bit 15
  > hltEG30LRId85ORIso60CaloId15b35eANDHE12R9Id50b80eLegCombLastFilter
hltEgammaCandidatesUnseeded > hltDiEG22EtEta2p55UnseededFilter
  > hltDiEG22R9Id50b80eR9IdUnseededFilter > hltDiEG22HE12R9Id50b80eHEUnseededFilter
  > ignore(hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter)           # bit 17
  > ignore(hltEG22CaloId15b35eHE12R9Id50b80eClusterShapeUnseededFilter)
  > ignore(hltEG22Iso60CaloId15b35eHE12R9Id50b80eEcalIsoUnseededFilter)
  > ignore(hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter)  # bit 16
```

The producers that compute the variables (hltEgammaR9IDUnseeded, hltEgammaHoverEUnseeded, hltEgammaHollowTrackIsoUnseeded, ...) sit between the filters and are omitted here. The same sequence, module for module, is in the 2018, 2022, 2023, 2024, 2025 and 2026 menus.

The two unseeded last filters: their candidate collection is explicit in l1EGCand

```
# bit 17 (v12 to v14: bit 15), trimmed to the relevant
parameters
fragment.hltEG22R9Id85b90eHE12R9Id50b80e
  R9UnseededLastFilter = cms.EDFilter(
  "HLTEgammaGenericFilter",
  saveTags = cms.bool( True ),
  candTag = cms.InputTag(
    "hltDiEG22HE12R9Id50b80eHEUnseededFilter" ),
  varTag = cms.InputTag(
    'hltEgammaR9IDUnseeded', 'r95x5' ),
  lessThan = cms.bool( False ),
  useEt = cms.bool( False ),
  thrRegularEB = cms.vdouble( 0.85 ),
  thrRegularEE = cms.vdouble( 0.9 ),
  ncandcut = cms.int32( 1 ),
  l1EGCand = cms.InputTag(
    "hltEgammaCandidatesUnseeded" ) )
```

```
# bit 16 (v12 to v14: bit 14), trimmed to the relevant
parameters
fragment.hltEG22Iso60CaloId15b35eHE12R9Id50b80e
  TrackIsoUnseededLastFilter = cms.EDFilter(
  "HLTEgammaGenericQuadraticFilter",
  saveTags = cms.bool( True ),
  candTag = cms.InputTag( "hltEG22Iso60CaloId15b35e
    HE12R9Id50b80eEcalIsoUnseededFilter" ),
  varTag = cms.InputTag(
    "hltEgammaHollowTrackIsoUnseeded" ),
  lessThan = cms.bool( True ),
  useEt = cms.bool( True ),
  thrRegularEB = cms.vdouble( 6.0 ),
  thrRegularEE = cms.vdouble( 6.0 ),
  thrOverEEB = cms.vdouble( 0.002 ),
  thrOverEEE = cms.vdouble( 0.002 ),
  ncandcut = cms.int32( 1 ),
  l1EGCand = cms.InputTag(
    "hltEgammaCandidatesUnseeded" ) )
```

candTag points to the previous filter of the chain (the objects to test), varTag to the variable producer, and l1EGCand to the candidate collection the filter tags its saved objects with. The seeded last filters are identical in structure with l1EGCand = hltEgammaCandidates. Module names are broken over two lines here only for display.

Why the unseeded objects carry type 92: the first filter of the leg saves them as TriggerCluster

```
# menu: the first unseeded filter
fragment.hltDiEG22EtEta2p55UnseededFilter =
  cms.EDFilter( "HLT1Photon",
  saveTags = cms.bool( True ),
  inputTag = cms.InputTag(
    "hltEgammaCandidatesUnseeded" ),
  triggerType = cms.int32( 92 ),
  MinE = cms.double( -1.0 ),
  MinPt = cms.double( 22.0 ),
  MinMass = cms.double( -1.0 ),
  MaxMass = cms.double( -1.0 ),
  MinEta = cms.double( -1.0 ),
  MaxEta = cms.double( 2.55 ),
  MinN = cms.int32( 2 ) )
```

```
// HLTrigger/HLTfilters/plugins/HLTSinglet.cc
template <typename T>
int getObjectType(const T&) { return 0; }
// no specialisation for RecoEcalCandidate

if (saveTags())
  filterproduct.addCollectionTag(inputTag_);
...
int tid = getObjectType<T>(*i);
if (tid == 0)
  tid = triggerType_;  // 92, TriggerCluster
filterproduct.addObject(tid, ref);
```

Result. Every candidate of hltEgammaCandidatesUnseeded with Et > 22 GeV is saved with type 92 and the collection tag hltEgammaCandidatesUnseeded. On the seeded leg, HLTEgammaL1TMatchFilterRegional and HLTEgammaEtFilter save TriggerCluster as well.

Why it matters. The NanoAOD selection starts with type(92). It is satisfied for both legs, so the object type is not what rejects the unseeded objects; the next slide shows what the later filters save, and the slide after that where the types are merged.

What the later filters save: HLTGenericFilter tags the collection and uses type 81 when saveTags is on

```
// HLTrigger/Egamma/plugins/HLTGenericFilter.cc (master lines 28 to 32 and 139 to 159; same logic from 10_6_X on)
l1EGTag_ = iConfig.template getParameter<edm::InputTag>("l1EGCand");
...
if (saveTags()) {
  filterproduct.addCollectionTag(l1EGTag_);  // hltEgammaCandidatesUnseeded for the unseeded leg
}
// Set output format
int trigger_type = trigger::TriggerCluster;
if (saveTags())
  trigger_type = trigger::TriggerPhoton;          // 81 for every saved object
...
PrevFilterOutput→getObjects(TriggerCluster, recoCands);
if (recoCands.empty())
  PrevFilterOutput→getObjects(TriggerPhoton, recoCands);  // accepts either type from the previous filter
```

Collection. The saved reference points into the l1EGCand collection. For the unseeded last filters that is hltEgammaCandidatesUnseeded, which PAT turns into the label hltEgammaCandidatesUnseeded::HLT on the object.

Type. These filters save type 81 (TriggerPhoton). The object keeps the 92 it received from the first filter, because PAT attaches the type of every filter that accepted an object (next slide). So type(92) in NanoAOD passes, and the collection test is the only thing left to fail.

Unchanged for years. The same three statements are in the 10_6_X, 12_4_X, 12_6_X, 13_0_X, 13_3_X, 14_0_X, 15_0_X and master versions of the plugin (only the line numbers move).

The trigger summary and PAT keep everything: collection label, all types, all filter labels

```
# menu: the summary producer
fragment.hltTriggerSummaryAOD =
  cms.EDProducer(
  "TriggerSummaryProducerAOD",
  throw = cms.bool( False ),
  processName = cms.string( "@" ),
  moduleLabelPatternsToMatch =
    cms.vstring( 'hlt*' ),
  moduleLabelPatternsToSkip =
    cms.vstring( ) )
```

Every filter with saveTags contributes; objects are stored once per collection tag, and each filter records the keys of its objects and the type it saved them with.

```
// PhysicsTools/PatAlgos/plugins/PATTriggerProducer.cc (master 680
to 731)
for (size_t iK = 0; iK < keys.size(); ++iK) {
  filterLabels.insert({keys[iK], nameFilter});
  objectTypes.insert({keys[iK], types[iK]});  // per filter
}
...
triggerObject.setCollection(
    handleTriggerEvent→collectionTagEncoded(iC));
for (auto t = objectTypes.equal_range(iO); ...)
  triggerObject.addTriggerObjectType(t.first→second);
...
triggerObjectStandAlone.addFilterLabel(frange.first→second);
triggerObjectStandAlone.addPathName(iP→pathName, ...);
```

Collection. setCollection stores the encoded tag of the collection the object came from: hltEgammaCandidatesUnseeded::HLT for the sub-leading leg, hltEgammaCandidates::HLT for the leading one.

Types and labels. addTriggerObjectType is called for every filter that referenced the object, so the unseeded object carries 92 and 81; addFilterLabel records every filter label, including the two unseeded last filters NanoAOD looks for.

MiniAOD slims the objects but keeps the collection name; NanoAOD unpacks the labels again

```
#
PatAlgos/python/slimming/selectedPatTrigger_cfi.py
selectedPatTrigger = cms.EDFilter(
  "PATTriggerObjectStandAloneSelector",
  src = cms.InputTag("patTrigger"),
  cut = cms.string("!filterLabels.empty()")
)
```

Keeps every object accepted by at least one filter with saveTags. The unseeded photons carry their last-filter labels, so they stay.

```
#
PatAlgos/python/slimming/slimmedPatTrigger_cfi.py
slimmedPatTrigger = cms.EDProducer(
  "PATTriggerObjectStandAloneSlimmer",
  src = cms.InputTag("selectedPatTrigger"),
  triggerResults =
    cms.InputTag( 'TriggerResults::HLT' ),
  packFilterLabels = cms.bool(True),
  packP4 = cms.bool(True),
)
```

Packs the four-vector and the filter labels; the collection string and the type list are stored as they are.

```
# NanoAOD/python/triggerObjects_cff.py,
lines 8 to 12
unpackedPatTrigger = cms.EDProducer(
  "PATTriggerObjectStandAloneUnpacker",
  patTriggerObjectsStandAlone =
    cms.InputTag('slimmedPatTrigger'),
  triggerResults =
    cms.InputTag('TriggerResults::HLT'),
  unpackFilterLabels = cms.bool(True)
)
```

Restores the labels so that filter('...') can be evaluated; hasFilterOrCondition throws if labels are still packed.

Proof that the unseeded objects really reach NanoAOD: the Electron selection has accepted coll('hltEgammaCandidatesUnseeded') since June 2023 and its bit "2e (CaloIdL_MW unseeded)" is populated in production files. The same objects enter the Photon branch of the producer and are rejected there.

The NanoAOD Photon selection: one collection in sel, three unseeded filters in the bit list

```
# PhysicsTools/NanoAOD/python/triggerObjects_cff.py, master lines 88 to 116 (v13: 81 to 108), abbreviated
Photon = cms.PSet(
  id = cms.int32(22),
  sel = cms.string("type(92) && pt > 15 && coll('hltEgammaCandidates')"),
  l1seed = cms.string("type(-98)"),  l1deltaR = cms.double(0.3),
  skipObjectsNotPassingQualityBits = cms.bool(True),
  qualityBits = cms.VPSet(
    mksel("filter('hltEG33L1EG26HEFilter')","hltEG33L1EG26HEFilter"),                  # bit 0
    ...  # bits 1 to 13: HE filters, hltHtEcal800, EB tight-ID filters, Mu17_Photon30 photon leg
    mksel("filter('hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter')", ...),              # 14
    mksel("filter('hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter')", ...),    # 15
    mksel("filter('hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter')", ...),  # 16
    mksel("filter('hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter')", ...),             # 17
    mksel("filter('hltEG30Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eEcalIsoFilter')", ...),  # 18
    mksel("filter('hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80e
           TrackIsoUnseededFilter')", ...),                                            # 19
    mksel("filter('hltEG*L1VBFLooseIsoEGHEFilter')", "hltEG*L1VBFLooseIsoEGHEFilter"),             # 20
  )
),
```

mksel turns a filter name into filter('name'), a list into an OR (max of the terms) and a tuple into an AND; bit i is the position in the VPSet unless bit= is given. No photon entry uses an explicit bit.

The contradiction is inside this one PSet: sel admits only hltEgammaCandidates, while bits 16, 17 and 19 name filters that run exclusively on hltEgammaCandidatesUnseeded (slide 7). No era modifier touches this block.

The table producer: sel first, bits second, duplicates merged last

```
// TriggerObjectTableProducer.cc, constructor,
// lines 105 to 118; cfg = qualityBitsConfig
for (size_t i = 0; i =/= cfg.size(); ++i) {
  unsigned int bit = i;
  if (cfg[i].existsAs<unsigned int>("bit"))
    bit = cfg[i].getParameter<unsigned int>("bit");
  assert(!bits[bit] && "...inserted twice");
  assert(bit < 64 && "...on 64 bits");
  func << std::to_string(1UL << bit) << "*("
       << cfg[i].getParameter<std::string>(
            "selection") << ")";
}
qualityBits = StringObjectFunction<...>(func.str());
```

The bits form one string function, the sum of 2^bit times each entry's selection. It is evaluated only for objects that already passed sel.

```
// produce(), master lines 151 to 176
for (const auto &obj : trigObjs) {
  for (const auto &sel : sels_) {
    if (sel.match(obj)) {  // fails here
      bits = sel.qualityBits(obj);
      if (skip ? (bits =/= 0) : true)
        selected.emplace_back(&obj, &sel);
    }
  }
}
// Self-cleaning
for (i ...) for (j < i ...)
  if (sel.id == sel2.id
      && deltaR2(obj, obj2) < 1e-6) {
    bits_of(obj2) |= bits_of(obj);
    selected.erase(selected.begin() + i); i--;
  }
```

Abridged: bits stands for selected_bits[sel.id][&obj], skip for sel.skipObjectsNotPassingQualityBits.

Order of evaluation is the whole story. An object that fails sel never reaches qualityBits, never enters selected, and is never seen by the merging loop. No later step can restore it.

No break. Since 13_0_X an object is tested against every selection and can be written once per type it passes (an e/gamma object can be both id 11 and id 22). 12_4_X still had a break after the first match.

The producer changed three times since 2022; the selection-before-bits order never did

releases (NanoAOD)	filterBits type	bit limit	duplicate merging	selections per object
12_4_X, 12_6_X (v10, v11)	Int_t	bit < 31	same id, same pT, same direction	first match only (break)
13_0_X to 14_2_X, 15_0_0, 15_0_1 (v12, v13, v14)	Int_t	bit < 31	same id, |delta pT| < 1e-6, delta R squared < 1e-6 (pT test dropped in 15_0_0)	every matching selection
15_0_2 onward, master (v15, next)	unsigned 64-bit	bit < 64	same id, delta R squared < 1e-6	every matching selection

```
// CMSSW_13_3_1_patch1 (v13), lines 114, 154, 170
assert(bit < 31 && "quality bits are store on 32
bit");
selected_bits[sel.id][&obj] =
int(sel.qualityBits(obj));
if (sel.id == sel2.id
    && abs(obj.pt() - obj2.pt()) < 1e-6
    && deltaR2(obj, obj2) < 1e-6) {
```

```
// master and CMSSW_15_0_2, lines 114, 154, 170
assert(bit < 64 && "quality bits are store on 64
bits");
selected_bits[sel.id][&obj] = sel.qualityBits(obj);
if (sel.id == sel2.id
    && deltaR2(obj, obj2) < 1e-6) {
```

What matters for this issue is in every version: sel.match(obj) is tested first, and only a passing object gets its quality bits computed. The merging rule only decides what happens after the fix, when a seeded and an unseeded object describe the same photon: with the master rule they merge on direction alone.

How coll('hltEgammaCandidates') is evaluated: no star, so an exact label comparison

```
// DataFormats/PatCandidates/src/TriggerObjectStandAlone.cc
bool TriggerObjectStandAlone::hasCollection(
    const std::string &collName) const {
  // Move to wild-card parser, if needed only
  if (collName.find(wildcard_) != std::string::npos) {
    if (hasAnyName(collName, {collection()}))
      return true;
    ...   // label / instance / process with wildcards
    return false;
  }
  // Use parent class's method otherwise
  return TriggerObject::hasCollection(collName);
}
```

```
// DataFormats/PatCandidates/src/TriggerObject.cc, 54 to 71
bool TriggerObject::hasCollection(
    const std::string& collName) const {
  if (collName == collection_) return true;
  const edm::InputTag collectionTag(collection_);
  const edm::InputTag collTag(collName);
  if (collTag.process().empty()) {
    if ((collTag.instance().empty()
        && collectionTag.instance().empty())
        || collTag.instance() == collectionTag.instance())
{
      return (collTag.label() == collectionTag.label());
    }
  }
  return false;
}
```

Evaluated for the unseeded object: collName = hltEgammaCandidates, collection_ = hltEgammaCandidatesUnseeded::HLT. No star, so the base method runs; labels hltEgammaCandidates and hltEgammaCandidatesUnseeded differ; the result is false.

Wildcards do exist (the Run 2 muon selection writes coll('hlt*L3MuonCandidates')), so coll('hltEgammaCandidates*') would also have worked. The explicit OR of the two names was chosen for the fix because it names exactly the two collections.

One unseeded photon object through the Photon selection, test by test

1 **type(92)**: true. hltDiEG22EtEta2p55UnseededFilter saved the object with triggerType 92 and PAT kept that type next to the 81 of the later filters.

2 **pt > 15**: true. MinPt is 22 GeV on this leg (18 GeV for the Diphoton30_18 paths).

3 **coll('hltEgammaCandidates')**: false. The label is hltEgammaCandidatesUnseeded; exact comparison (previous slide).

4 **sel.match(obj) is false**: qualityBits is never evaluated; the object is not in selected; bits 16 and 17 are 0 in every event, in every file.

```
# the Electron selection, same file
# CMSSW_13_0_13 (v12)
sel = "type(92) && pt > 7
  && coll('hltEgammaCandidates')
  && filter('*PixelMatchFilter')"

# CMSSW_13_3_0 onward (v13), PR 41827
sel = "type(92) && pt > 7
  && (coll('hltEgammaCandidates')
      || coll('hltEgammaCandidatesUnseeded'))
  && (filter('*PixelMatchFilter')
      || filter('*PixelMatchUnseededFilter'))"
```

PR 41827 (merged 8 June 2023) added the unseeded CaloIdL_MW electron bit and "added the collection hltEgammaCandidatesUnseeded, which are the objects on which the selection of the unseeded trigger leg is made". The photon diphoton bits had been added four months earlier by PR 40711 without that change, and nobody came back to it.

Six other explanations, and the evidence that rules each one out

hypothesis	what the code and the menus show	verdict
The filter names in NanoAOD are misspelt or outdated	All four Diphoton30_22 names are defined with saveTags = True in the 2018 GRun, 2022 v1.2/v1.4/v1.5, 2023, 2024, 2025 and 2026 menus and in every development menu checked (19 files); the chain is identical everywhere	ruled out
The unseeded objects never reach MiniAOD	patTrigger excludes no collection; selectedPatTrigger keeps objects with any filter label; the Electron selection has consumed coll('hltEgammaCandidatesUnseeded') in production since June 2023	ruled out
The objects have the wrong type for type(92)	HLT1Photon saves them with triggerType 92; the later filters add 81; PAT keeps both types	ruled out
The pT threshold removes them	sel requires pt > 15; the unseeded leg requires Et > 22 GeV (18 GeV for Diphoton30_18)	ruled out
An era modifier changes the Photon block for Run 3	The modifiers in triggerObjects_cff.py act on Muon, Tau and Jet (Run 2 eras) and on the prefiring producer; none touches Photon	ruled out
Something specific to the v13 release	CMSSW_13_3_1_patch1 and the 13_3_X tip have byte-identical NanoAOD configuration; 13_0_X, 14_0_X, 15_0_X, 16_x and master carry the same Photon selection	ruled out
The Photon selection rejects the collection	sel = type(92) && pt > 15 && coll('hltEgammaCandidates'), exact comparison, evaluated before the bits	the cause

Each row was checked in the source: menu files (filter definitions and saveTags), PhysicsTools/PatAlgos (patTrigger, selectedPatTrigger, slimmedPatTrigger), the HLT plugins (HLTSinglet, HLTGenericFilter), the era modifiers at the end of triggerObjects_cff.py, and the configuration at the tags and branch tips named on the next slide.

Scope by NanoAOD version: present since the bits were added, still present in today's master

NanoAOD	releases checked (tags and branch tips)	Photon sel	Electron sel	dead photon bits	affected
v9	CMSSW_10_6_X	seeded, pt > 20, no bits	seeded	no photon TrigObj	no
v10, v11	CMSSW_12_4_X, CMSSW_12_6_X	seeded, pt > 20, 12 bits	seeded	bits absent	no
v12	13_0_0 (pt > 20), 13_0_1 to 13_0_17 (prod. 13_0_13)	seeded, 18 bits	seeded	14, 15, 17	yes
v13	13_3_0, 13_3_1_patch1 (production), 13_3_X tip	seeded, 18 bits	seeded + unseeded	14, 15, 17	yes
v14	14_0_0, 14_0_X tip, 14_1_0, 14_1_X, 14_2_0, 14_2_X	seeded, 18 bits	seeded + unseeded	14, 15, 17	yes
v15	15_0_2 to 15_0_15_patch4, 15_0_X tip, 15_1_0, 15_1_X (15_0_0 and 15_0_1 still 18 bits)	seeded, 21 bits	seeded + unseeded	16, 17, 19	yes
next	16_0_10, 16_0_X, 16_1_3, 16_1_X, master ed35654b1 (4 Nov 2026, newest commit)	seeded, 21 bits	seeded + unseeded	16, 17, 19	yes

Checked live today. git ls-remote on cms-sw/cmssw gives master = ed35654b1 (no newer commit), 16_1_X = 8aacfa5da, 16_0_X = de8bdeed9, 15_1_X = e9272496, 15_0_X = 6db81eb6d; the Photon selection line is identical on all of them and at the newest tags CMSSW_16_1_3 and CMSSW_16_0_10.

Nobody has fixed it elsewhere. GitHub shows no open pull request or issue mentioning hltEgammaCandidatesUnseeded, triggerObjects_cff or photon trigger objects, and no change to triggerObjects_cff.py since June 2026.

Confirmed on twelve NanoAOD files, v9 to v15: the unseeded photon bits are never set

NanoAOD	file (one per dataset, read on lxplus)	events	fired Mass90	photons: seeded bits	unseeded bits	e unseeded	id 2	id 1515
v9	UL2018 data, EGamma 2018D	300000	31771	no photon TrigObj	none defined	n/a	0	0
v10	2022 prompt data, EGamma 2022D	300000	46707	bits not defined	none defined	n/a	0	0
v11	2022 prompt data, EGamma 2022F	300000	41546	bits not defined	none defined	n/a	0	0
v12	Summer22 MC, GluGluHtoGG M125	12979	7725	15626 / 17978	0 / 0 / 0	n/a	0	0
v12	2022 data, EGamma 2022D 22Sep2023	300000	46407	208527 / 255099	0 / 0 / 0	n/a	0	0
v13	Summer23 MC, GluGluHto2G M125	17760	10453	20865 / 24170	0 / 0 / 0	702	0	0
v13	2022 data, EGamma 2022D 16Dec2023	300000	46794	207844 / 252774	0 / 0 / 0	29042	0	0
v14	2024 prompt data, EGamma0 2024D	300000	35892	203620 / 252039	0 / 0 / 0	19444	0	0
v15	Summer24 MC, GluGluH to 2G M125	4180	2406	4815 / 5588	0 / 0 / 0	205	0	22
v15	2022 data, EGamma 2022D NanoAODv15	300000	48687	212270 / 258546	0 / 0 / 0	30451	0	430
v15	2024 data, EGamma0 2024D MINIv6NANOv15	300000	30945	201628 / 248974	0 / 0 / 0	22980	0	254
133X	Winter24 MC, GluGluHToGG M125 powheg	175654	101598	205090 / 239250	0 / 0 / 0	9860	0	0

Controls behave. v9 has no photon objects; v10 and v11 document no diphoton bits; the electron unseeded bit is absent in v12 and populated from v13.

Affected versions behave. Seeded bits on more objects than Mass90 events; unseeded bits 0 in all 1.7 million events of v12 to v15 and Winter24 (133X).

Side findings behave.
No id 2 (MET) object in any file; id 1515 (BoostedTau) only in v15 (22, 430, 254). Fix re-run: next slide.

Fix test on real input: every Mass90 event gets its sub-leading-leg bits, nothing else changes

NANO file (17760 events each)	fired Mass90	photons: seeded bits	photons: unseeded bits	Mass90 events with an unseeded bit	nTrigObj mean	size (bytes)
central v13 file	10453	20865 / 24170	0 / 0 / 0	0	5.287	29333713
v13 re-made, no fix	10453	20865 / 24170	0 / 0 / 0	0	5.287	29339920
v13 re-made, with fix	10453	20865 / 24170	20461 / 18365 / 0	10453 (all)	5.388	29363499
v15 re-made, no fix	10453	20865 / 24170	0 / 0 / 0	0	38.134	39947152
v15 re-made, with fix	10453	20865 / 24170	20461 / 18365 / 0	10453 (all)	38.225	39959608

- Re-made without the fix, the file is identical to the central v13 file: same TrigObj (id, bits, pt) in 17760 of 17760 events.
- The 21254 photon objects that gain an unseeded bit have pt from 22.04 GeV (median 58.30 GeV), none below the 22 GeV leg threshold.
- Every object type other than photons is unchanged in 17760 of 17760 events; photon objects 24835 to 26639 (v13), 25188 to 26807 (v15).
- The third unseeded bit (Diphoton30_18 leg) stays 0: that filter name does not exist in the 2023 menu.
- Relval in CMSSW_16_1_3 with the fix (12452.0, 141.046): builds and runs without exceptions; outputs identical to the clean release except 25 DQM TimerService histograms per workflow. Its NANO TrigObj table is empty in both areas, so it does not exercise the photon selection.

Seeded / unseeded bits: 12, 13 / 14, 15, 17 in v13; 14, 15 / 16, 17, 19 in v15. Input: the MiniAOD parent of the central Run3Summer23 v13 GluGluHto2G M125 file; CMSSW_13_3_1_patch1 (v13) and CMSSW_15_0_15_patch4 (v15). 5 November 2026, results/REPORT.md "Fix test" and "Relval in CMSSW_16_1_3".

Scope by HLT menu: nine years of menus, the same four filters on the same collections

menu (table name)	Mass90/95	Diphoton30_18	unseeded photon bits dead (v12 to v14 / v15 numbering)
2018 GRun, /dev/CMSSW_10_6_0/GRun/V3	v13	NoPixelVeto v2	14, 15, 17 / 16, 17, 19
2022 frozen v1.2 (HLT_2022v12, Run3Summer22 MC)	v13	v2	14, 15, 17 / 16, 17, 19
2022 frozen v1.4 and v1.5, GRun 12_4 V171, GRun 12_6 V16	v15	v3	14, 15, 17 / 16, 17, 19
2023 frozen v1.2 (HLT_2023v12), GRun 13_0 V153	v17	v5 (renamed filters)	14, 15 / 16, 17
2023 development, 13_3 GRun V11 and HLT V19	v19	v7	14, 15 / 16, 17
2024 frozen v1.4 (HLT_2024v14), GRun 14_0 V182	v22	v10	14, 15 / 16, 17
2025 development (fork, May 2025), GRun 15_0 V62	v24	v12	16, 17
2025 frozen v1.3 (HLT_2025v13), GRun 15_0 V120	v26	v14	16, 17
2026 frozen v1.3 (HLT_2026v13, in master)	v29	v17	16, 17
2026 development, GRun 16_1 V16 (master)	v30	v18	16, 17

In all of these the two unseeded last filters of the Mass90/Mass95 sequence carry l1EGCand = hltEgammaCandidatesUnseeded and saveTags = True, and the ordered list of the sixteen e/gamma filters of the sequence is identical (same MD5). From 2023 the Diphoton30_18 filters were renamed, so its two NanoAOD bits (18, 19) match nothing in those menus, a separate stale-name problem.

How widespread: a scanner that resolves every filter bit of every object type against each menu

```
# scripts/deadbits.py (notes repository), the core
def glob_re(pat):  # NanoAOD wildcard semantics
  parts = pat.split("*")
  return re.compile("^" + ".*".join(
      re.escape(p) for p in parts) + "$")

def collections_of(mods, name):
  # EDProducer labels a filter saves its objects from
  typ, body = mods[name]
  for p in ("l1EGCand", "inputTag", "CandTag",
            "inputCandCollection", "originTag1", ...):
    for lab in tag_labels(body, p):
      if mods[lab][0] == "EDProducer": direct.add(lab)
      elif mods[lab][0] == "EDFilter": follow.append(lab)
  return direct or union(collections_of(mods, f) for f in follow)

status = OK if any matched filter's collection passes coll()
       else COLL if filters match but none passes
       else STALE if no module matches the pattern
```

Inputs

Configs: master, 15_0_X (v15), 13_3_1_patch1 (v13), 13_0_13 (v12). Menus: frozen 2022 v1.2 and v1.4, 2023 v1.2, 2024 v1.4, 2025 v1.3, 2026 v1.3, plus the GRun development menus of 13_0_X, 14_0_X, 15_0_X and 16_1_X.

What it emulates

mksel (lists are ORs, tuples ANDs), the anchored star matching of hasAnyName, the collection each matched filter saves (following inputTag chains for combination filters), and the coll() terms of each selection. The type test is not emulated; the Tau selection's coll('*Tau*') is OR-ed with filters and treated as non-restrictive.

Outputs

Per config, selection, bit and menu: OK, COLL, STALE or UNK (L1-seed modules, accepted by type). Raw outputs are in scan/ of the notes repository.

Scan results: the collection mismatch is unique to the Photon list; stale names are everywhere

object	bits	never settable (collection or type)	fine / stale, 2024 menu	fine / stale, 2025 menu	stale in 2025, why
Electron	20	none	19 / 1	18 / 2	WPLoose (gone 2023), triple-electron (gone 2025)
Photon	21	bits 16, 17, 19 (unseeded) and 8 (hltHtEcal800)	15 / 3	15 / 3	bit 12 (gone 2022), 18 and 19 (renamed 2023)
Muon	14	none	14 / 0	12 / 2	mu-tau DeepTau overlaps (gone 2025)
Tau	31	none	31 / 0	30 / 1	DeepTau (gone 2025)
BoostedTau	3	v12 to v14: the whole selection (next slide)	2 / 1	2 / 1	2022 ParticleNetTauTau name
Jet	46	none	37 (+2 L1) / 7	34 (+2 L1) / 10	pixel-only HH4b filters (gone 2024), DeepCSV and DeepJet (gone 2025), 3-jet pt40 and pt45
FatJet	7	none	4 / 3	4 / 3	2022 and 2023 soft-drop and ParticleNet names
HT, MHT	6, 2	none	3 (+1 L1) / 2, 2 / 0	same	two Run 2 L1 seed names, never in Run 3 menus
MET	0	no bits with the skip flag on: never written	none	none	all versions since v9

Same mechanism as the diphoton bits: only Photon bit 8, whose filter saves HT objects. Electron and Muon have no collection mismatch in any version or menu.

Stale names are a documentation problem: one bit list serves the 2022 to 2026 menus, so bits for renamed or retired filters stay documented but silently empty for later years.

Three more entries that can never be filled, found by the same scan

Photon bit 8: hltHtEcal800

```
fragment.hltHtEcal800 =
cms.EDFilter(
  "HLTHtMhtFilter",
  saveTags = cms.bool( True ),
  htLabels = cms.VInputTag(
    'hltHtMhtEcal' ),
  minHt = cms.vdouble( 800.0 ),
... )

// HLTHtMhtFilter.cc, 104 to 105
addObject(trigger::TriggerTHT,
htref);
addObject(trigger::TriggerMHT,
mhtref);
```

HT objects (types 89, 90) from hltHtMhtEcal: never type 92, never an e/gamma collection. Dead since the bit appeared in the 12_4_X list (v10); the v9 photon list had no bits at all.

MET: no bits, skip flag on

```
MET = cms.PSet(
  id = cms.int32(2),
  sel = cms.string("type(87) && pt
> 30
    && coll('hltPFMETProducer')"),
  skipObjectsNotPassingQualityBits
    = cms.bool(True),
  qualityBits = cms.VPSet()
)
```

With no entries the bit function is the constant "0"; with the skip flag every MET object is dropped. Identical in 10_6_X (qualityBits "0"), 12_4_X, v12 to v15 and master: TrigObj has never contained MET entries although the documentation lists id 2.

BoostedTau in v12 to v14: empty table

```
# 13_0_X to 15_0_1
sel = "type(85) && pt > 120
  && coll('hltAK8PFJetsCorrected')
  && filter('hltAK8SinglePFJets*
     SoftDropMass40*ParticleNetTauTau')"

# menus: 2022 ...ParticleNetTauTau0p30
#        2023+ ...PNetTauTauTag0p03
# 15_0_2: filter('*TauTau*')
```

The pattern is anchored at its end, so no menu name ever matched, and the jets come from hltAK8PFJets230SoftDropMass40, not hltAK8PFJetsCorrected. Relaxed in CMSSW_15_0_2.

Stale bits by year: documented filters that no longer exist in the menu of that year

object	bit	filter pattern (master list)	exists in menus	empty for data of
Electron	2	hltEle*WPLoose*TrackIsoFilter	2022	2023 onward
Electron	8	hltEle*Ele*Ele*CaloIdLTrackIdLDphiLeg*Filter (3e)	2022 to 2024	2025 onward
Photon	12	hltEG120EBTightIDTightIsoTrackIsoFilter	2018, early 2022	2022 v1.4 onward
Photon	18, 19	hltEG30Iso60CaloId15b35eR9Id50b90eHE12b10e… and hltEG18TrackIso60… (Diphoton30_18)	2018, 2022	2023 onward (renamed)
Muon	2, 6	*OverlapFilterIsoMu*PFTau* (DeepTau mu-tau overlap)	2022 to 2024	2025 onward (bit 13 PNet replaces)
Tau	3	*DeepTau*	2022 to 2024	2025 onward (bit 4 PNet replaces)
BoostedTau	0	hltAK8SinglePFJets*SoftDropMass*ParticleNetTauTau0p30	2022	2023 onward (bits 1, 2 replace)
Jet	0, 1, 6, 22, 27	hlt{4,3,2,1}PixelOnlyPFCentralJetTightIDPt{20,30,40,60} (HH to 4b)	2022, 2023	2024 onward
Jet	8, 9, 11, 24, 25	hlt3PFCentralJetTightIDPt40/45, hltBTagCaloDeepCSVp17Double, hltBTagPFDeepJet1p5Single/4p5Triple	2022 to 2023/2024	2023, 2024 or 2025 onward
FatJet	3, 4, 5	…SoftDropMass40BTagParticleNetBB0p35, hltAK8DoublePFJetSDModMass30/50	2022 (and 2023 for 4, 5)	2023 or 2024 onward
HT	0, 1	hltL1sTripleJetVBFIorHTTIorDoubleJetCIorSingleJet, hltL1sQuadJetC50Ior… (Run 2 L1 seeds)	Run 2 only	all Run 3

Bits added for newer filters (for example Photon 9, 10, 20; Electron 17, 19; Tau PNet entries; Jet 28 to 41) are empty for the earlier years in the same way. None of this is a code defect; it is the price of one bit list for five years of menus, and the documentation string does not say which years a bit covers.

What analyses see: every Run 3 NanoAOD file, and no sub-leading match for the diphoton triggers

Datasets

Data: 2022 (PromptNano v10/v11, 22Sep2023 v12, 16Dec2023 v13), 2023 (22Sep2023 v12), 2024 (prompt v14), 2025 (prompt v15) and the v15 re-NANO of 2022 to 2024. MC: Run3Summer22, 22EE, 23, 23BPix at v12 and v13, Summer24 at v15, and the v15 re-NANO of all of them. Any Run 2 input re-processed with v12 or later.

Frameworks

NanoAOD-based photon matching (HiggsDNA-style workflows, the HYgammaRunIII legend that lists bits 16 and 17 as the legs, FLAF configurations, custom NanoAOD setups) finds no object for the sub-leading leg. MiniAOD-based flashgg reads the pat trigger objects directly and is unaffected.

Existing files cannot be repaired. The unseeded objects were never written; no post-processing can recover them. Only a new NANO pass with the fix restores the information.

```
# schematic TrigObj rows of one diphoton-triggered event
# (illustration of the structure, not real data)
today (v15):
 id  pt     filterBits      meaning
 22  41.7   0x4000 (bit 14)  leading leg, R9 last
 -- no row for the 26 GeV photon --

with the fix:
 22  41.7   0x4000 (bit 14)  leading leg, R9 last
 22  26.3   0x20000 (bit 17) unseeded leg, R9 last
```

A leading photon that also passes the unseeded leg merges with its unseeded twin (same direction) and carries both bits; a sub-leading photon appears as a new row.

The fix: accept both e/gamma collections in the Photon selection

```
diff --git a/PhysicsTools/NanoAOD/python/triggerObjects_cff.py b/PhysicsTools/NanoAOD/python/triggerObjects_cff.py
@@ -87,7 +87,7 @@ triggerObjectTable = triggerObjectTableProducer.clone(
         Photon = cms.PSet(
             id = cms.int32(22),
-            sel = cms.string("type(92) && pt > 15 && coll('hltEgammaCandidates')"),
+            sel = cms.string("type(92) && pt > 15 && (coll('hltEgammaCandidates') ||
coll('hltEgammaCandidatesUnseeded'))"),
             l1seed = cms.string("type(-98)"),  l1deltaR = cms.double(0.3),
             skipObjectsNotPassingQualityBits = cms.bool(True),
```

Why this form

Names exactly the two collections and copies the Electron selection of PR 41827. A wildcard would also match (next slide) but would admit any future collection with that prefix.

What changes

Unseeded objects enter the photon branch; with the skip flag only those that set a photon bit are written, and the only photon-bit filters on the unseeded collection are the three diphoton ones. A seeded twin in the same direction is merged with OR-ed bits.

What does not

Bit numbers, documentation strings, the other object types and the event content. Only events that fired a diphoton trigger gain photon rows.

Status. Committed as 1699a39f7 on the fork branch claude/fervent-lamport-3oiuh6 and as aebf3eb64 on branch nano-photon-trigobj-unseeded on top of upstream master ed35654b1. Both branches are on the fork; nothing pushed to cms-sw/cmssw, no pull request opened.

Why not a wildcard, and exactly which objects the fix lets through

```
// TriggerObjectStandAlone::hasAnyName, lines 80 to 119 (abridged)
boost::split(parts, name, is_any_of("*"), token_compress_on);
for (name in nameVec) {
  index = 0;
  for (part in parts) {
    if (part.empty()) continue;
    index = name.find(part, index);
    if (index == npos ||
        (part is first && index > 0)) { failed; break; }
    index += part.length();
  }
  // Failed, if end of name not reached
  if (index < name.length() && !parts.back().empty())
    failed = true;
}
```

coll('hltEgammaCandidates*') would match both collections. The anchoring rule is also what kills the v12 to v14 BoostedTau pattern (no trailing star, name continues).

unseeded filter that is a photon bit	bit	menus
hltEG22Iso60CaloId15b35eHE12R9Id50b80e TrackIsoUnseededLastFilter	16	2018 to 2026
hltEG22R9Id85b90eHE12R9Id50b80e R9UnseededLastFilter	17	2018 to 2026
hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90e HE12b10eR9Id50b80eTrackIsoUnseededFilter	19	2018, 2022

These are the only entries of the photon list whose filters run on hltEgammaCandidatesUnseeded (checked in nine menus; the wildcard bit 20 matches only the seeded hltEG22L1VBFLooseIsoEGHEFilter). Every other unseeded e/gamma object, for example from electron pixel-match legs, sets no photon bit and is dropped by the skip flag. The output grows only where these three filters fired.

Validation: relval workflows before and after, and a direct count of the new bits

1. Standard workflows with a NANO step

Done in CMSSW_16_1_3 (5 November): 12452.0 (10 events) and 141.046 (100 events). No exceptions; identical to the clean release except DQM timing histograms. The relval NANO has no TrigObj, so the photon selection is not exercised.

2. Differences seen in the fix test

NANO re-made from one MiniAOD file (17760 events): unseeded bits in all 10453 Mass90 events (zero before), pt from 22.04 GeV; every other object type identical in all events.

3. Already measured on files

Twelve NanoAOD files, v9 to v15 (4-5 November): unseeded photon bits 0 in all of them, all controls as predicted (results/REPORT.md).

```
# count photon objects with the unseeded bits, per path
import uproot, awkward as ak, numpy as np
t = uproot.open("nano.root")["Events"]
path = "HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90"
a = t.arrays(["TrigObj_id", "TrigObj_filterBits", path])
ph    = a["TrigObj_id"] == 22
bits  = ak.values_astype(a["TrigObj_filterBits"][ph],
np.uint64)
fired = a[path]
for bit in (16, 17):   # 14, 15 for NanoAOD v12 to v14
  has = ak.any(((bits >> np.uint64(bit)) & np.uint64(1)) == 1,
               axis=1)
  print(bit, int(np.sum(has & fired)), "of",
int(np.sum(fired)))
```

Before the fix both counts are 0 by construction. After it they must be of the order of the number of events that fired the path. Measured in the fix test: every one of the 10453 Mass90 events has a photon with at least one of the two bits.

Pull request plan, current status and the follow-ups kept out of the one-line fix

```
# pull request (prepared, not opened)
title:  NanoAOD: accept unseeded e/gamma trigger
        objects in the Photon TrigObj selection
from:   santanumahata:nano-photon-trigobj-unseeded
        (aebf3eb64 on top of master ed35654b1)
to:     cms-sw:master, xpog signature
cites:  PR 40711 (bits added), PR 41827 (electrons
fixed)
backport: CMSSW_15_0_X if another production is
planned
```

- Status: the PR branch nano-photon-trigobj-unseeded (aebf3eb64) and the fork branch are on santanumahata/cmssw; no pull request exists; cms-sw/cmssw is untouched.
- Next action is yours: open the pull request from the GitHub web interface.

Follow-ups, each a separate pull request

- Add the 2023 names of the Diphoton30_18 filters as OR alternatives inside bits 18 and 19 (keeps the numbering stable; the unseeded one needs this collection fix to work).
- Move bit 8 (hltHtEcal800) out of the Photon list, or drop it: its objects are HT objects.
- State in the documentation string which menus each bit covers, and review the stale bits per year (WPLoose, triple-electron, DeepTau overlaps, pixel-only jet filters, DeepCSV, DeepJet).
- Photon bits for the low-mass eta1p5, DiphotonMVA14p25, Diphoton15_1x EBEB and DoublePhoton paths, with the EGM POG and the Higgs PAG.
- Tell XPOG that MET trigger objects are never written and that the BoostedTau table was empty in v12 to v14.

Appendix A: photon filter bits 0 to 10 (master numbering) and where their filters exist

bit v15	bit v12 to v14	filter	objects come from	filter exists in menus
0	0	hltEG33L1EG26HEFilter	hltEgammaCandidates	2018 to 2026
1	1	hltEG50HEFilter	hltEgammaCandidates	2018 to 2026
2	2	hltEG75HEFilter	hltEgammaCandidates	2018 to 2026
3	3	hltEG90HEFilter	hltEgammaCandidates	2018 to 2026
4	4	hltEG120HEFilter	hltEgammaCandidates	2018 to 2026
5	5	hltEG150HEFilter	hltEgammaCandidates	2018 to 2026
6	6	hltEG175HEFilter	hltEgammaCandidates	2018 to 2026
7	7	hltEG200HEFilter	hltEgammaCandidates	2018 to 2026
8	8	hltHtEcal800	hltHtMhtEcal (HT object, type 89): never a photon	2018 to 2026
9	none	hltEG45EBTightIDTightIsoTrackIsoFilter	hltEgammaCandidates	2024 to 2026
10	none	hltEG50EBTightIDTightIsoTrackIsoFilter	hltEgammaCandidates	2023 to 2026

Source: PhysicsTools/NanoAOD/python/triggerObjects_cff.py at master ed35654b1 (21 entries) and CMSSW_13_3_1_patch1 (18 entries); menu presence from the frozen 2018 to 2026 menus listed on the menus slide. "Objects come from" is the l1EGCand (or htLabels) of the module in the menu.

Appendix B: photon filter bits 11 to 20, with the three dead unseeded entries

v15	v12 to v14	filter	objects come from	filter exists in
11	9	`hltEG110EBTightIDTightIsoTrackIsoFilter`	hltEgammaCandidates	2018 to 2026
12	10	`hltEG120EBTightIDTightIsoTrackIsoFilter`	hltEgammaCandidates	2018, 2022 v1.2 only
13	11	`hltMu17Photon30IsoCaloIdPhotonlegTrackIsoFilter`	hltEgammaCandidates	2018 to 2026
14	12	`hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter`	hltEgammaCandidates	2018 to 2026
15	13	`hltEG30LIso60CaloId15b35eHE12R9Id50b80e EcalIsoLastFilter`	hltEgammaCandidates	2018 to 2026
16	14	`hltEG22Iso60CaloId15b35eHE12R9Id50b80e TrackIsoUnseededLastFilter`	hltEgammaCandidatesUnseeded, never set	2018 to 2026
17	15	`hltEG22R9Id85b90eHE12R9Id50b80e R9UnseededLastFilter`	hltEgammaCandidatesUnseeded, never set	2018 to 2026
18	16	`hltEG30Iso60CaloId15b35eR9Id50b90e HE12b10eR9Id50b80eEcalIsoFilter`	hltEgammaCandidates	2018, 2022 (renamed 2023)
19	17	`hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90e HE12b10eR9Id50b80eTrackIsoUnseededFilter`	hltEgammaCandidatesUnseeded, never set	2018, 2022 (renamed 2023)
20	none	`hltEG*L1VBFLooseIsoEGHEFilter (only hltEG22L1VBFLooseIsoEGHEFilter)`	hltEgammaCandidates	2023 to 2026

Long module names are broken over two lines for display only. Decimal values of the dead bits: 65536, 131072, 524288 (v15 and master); 16384, 32768, 131072 (v12 to v14).

Appendix C: how the defect was born, survived and was found

7 to 8 Feb 2023

PR 40711 adds the six diphoton photon bits for the next NanoAOD production. The Photon selection is not touched, so the three unseeded bits are dead from day one. In CMSSW_13_0_0; v12 is made with 13_0_13.

31 May to 8 Jun 2023

PR 41827 gives the Electron selection the unseeded collection and the CaloIdL_MW unseeded bit. Photons are not revisited. In CMSSW_13_3_0, so NanoAOD v13 has it for electrons only.

Nov to Dec 2023

13_3_X backports for v13 (43292 from 43247, 43286, 43470 and others); CMSSW_13_3_1_patch1 produces NanoAOD v13 for Summer22, 22EE, 23, 23BPix MC and the 16Dec2023 2022 data.

13 to 18 Mar 2025

PR 47594 / 47601: hltEG45 and hltEG50 EB bits and the VBF wildcard are added and filterBits becomes 64-bit (CMSSW_15_0_2). The unseeded bits move to 16, 17 and 19, still dead. v15 inherits them.

4 Nov 2026

This investigation: cause located, confirmed on master ed35654b1 and the 16_x branches, whole-table scan done, one-line fix committed on top of upstream master, pull request text ready. No open upstream PR or issue on the subject.

Dates are the creation and merge dates returned by the GitHub search API. Between CMSSW_13_0_0 and 13_0_1 the photon pt threshold moved from 20 to 15 GeV; that change did not touch the collection test either.

Backup code: verify_nanoaod_files.py (1/5)

scripts/verify_nanoaod_files.py, lines 1 to 34

```
#!/usr/bin/env python3
"""Empirical check of the dead unseeded-leg photon TrigObj bits in CMS NanoAOD files.

usage:
  python3 verify_nanoaod_files.py --label v13 --files root://cms-xrd-global.cern.ch//store/... [more files]
                                  [--max-events 300000] [--out results_v13.json]

For every file it reads the TrigObj_filterBits documentation (branch title) to find the bit numbers
of the relevant filters in THAT file (so v12 to v14 and v15 numbering are both handled), then counts:
  - events firing the diphoton paths (and the DoubleEle CaloIdL_MW paths, electron control),
  - photon trigger objects (id 22) with the seeded last-filter bits (expected > 0),
  - photon trigger objects with the unseeded last-filter bits (the claim: always 0 before the fix),
  - photon objects with the hltHtEcal800 bit (claim: always 0),
  - electron objects (id 11) with the "CaloIdL_MW unseeded" bit (control: > 0 from v13 on),
  - trigger objects per id (MET id 2 expected 0 in every version; BoostedTau id 1515 expected 0 in v12 to v14).
Needs python3 with uproot, awkward, numpy (LCG view or a CMSSW environment) and xrootd access for root:// files.
"""
import argparse, json, re, sys, time
import numpy as np
import awkward as ak
import uproot

UNSEEDED = {
    "TrackIsoUnseededLast": "hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter",
    "R9UnseededLast": "hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter",
    "EG18TrackIsoUnseeded": "hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter",
}
SEEDED = {
    "R9IdLast": "hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter",
    "EcalIsoLast": "hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter",
    "EG30EcalIso3018": "hltEG30Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eEcalIsoFilter",
}
HT_BIT = "hltHtEcal800"
ELE_UNSEEDED_DOC = "CaloIdL_MW unseeded"
```

Backup code: verify_nanoaod_files.py (2/5)

scripts/verify_nanoaod_files.py, lines 35 to 61

```
PATHS = [
    "HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90",
    "HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass95",
    "HLT_Diphoton30_18_R9IdL_AND_HE_AND_IsoCaloId",
    "HLT_Diphoton30_18_R9IdL_AND_HE_AND_IsoCaloId_Mass55",
    "HLT_DoubleEle33_CaloIdL_MW",
    "HLT_DoubleEle25_CaloIdL_MW",
]
IDS = {2: "MET", 11: "Electron", 22: "Photon", 13: "Muon", 15: "Tau", 1515: "BoostedTau", 1: "Jet", 6: "FatJet", 3: "HT", 4: "MHT"}


def parse_bitdoc(title):
    """'extra bits ...: 0 ⇒ a, 1 ⇒ b for Electron; 0 ⇒ c for Photon; ...' → {object: {bit: doc}}"""
    out = {}
    body = title.split(":", 1)[1] if ":" in title else title
    for chunk in body.split("; "):
        m = re.search(r"\s+for\s+(\w+)\s*$", chunk.strip())
        if not m:
            continue
        obj, entries = m.group(1), chunk.strip()[: m.start()]
        d = {}
        for e in re.finditer(r"(\d+)\s*⇒\s*(.*?)(?=,\s*\d+\s*⇒|$)", entries):
            d[int(e.group(1))] = e.group(2).strip()
        out[obj] = d
    return out
```

Backup code: verify_nanoaod_files.py (3/5)

scripts/verify_nanoaod_files.py, lines 62 to 95

```
def find_bit(docs, needle):
    for bit, doc in docs.items():
        if needle in doc:
            return bit
    return None


def has_bit(bits, bit):
    return ((bits >> np.uint64(bit)) & np.uint64(1)) == 1


def check_file(path, max_events):
    t0 = time.time()
    f = uproot.open(path)
    t = f["Events"]
    keys = set(t.keys())
    title = t["TrigObj_filterBits"].title
    dtype = str(t["TrigObj_filterBits"].interpretation)
    docs = parse_bitdoc(title)
    pho, ele = docs.get("Photon", {}), docs.get("Electron", {})
    bits_unseeded = {k: find_bit(pho, v) for k, v in UNSEEDED.items()}
    bits_seeded = {k: find_bit(pho, v) for k, v in SEEDED.items()}
    bit_ht = find_bit(pho, HT_BIT)
    bit_ele_unseeded = find_bit(ele, ELE_UNSEEDED_DOC)
    paths = [p for p in PATHS if p in keys]
    branches = ["TrigObj_id", "TrigObj_filterBits"] + paths
    res = {
        "file": path, "events": 0, "filterBits_dtype": dtype, "n_photon_bits_documented": len(pho),
        "bits_unseeded": bits_unseeded, "bits_seeded": bits_seeded, "bit_hltHtEcal800": bit_ht,
        "bit_electron_caloIdL_MW_unseeded": bit_ele_unseeded,
        "paths_present": paths, "fired": {p: 0 for p in paths},
        "objects_per_id": {}, "photon_objects_with_bit": {}, "events_fired_mass90_with_photon_bit": {},
        "electron_objects_with_unseeded_bit": 0,
    }
```

Backup code: verify_nanoaod_files.py (4/5)

scripts/verify_nanoaod_files.py, lines 96 to 129

```
    allbits = {**{"unseeded:" + k: v for k, v in bits_unseeded.items()}, **{"seeded:" + k: v for k, v in bits_seeded.items()}, "hltHtEcal800": bit_ht}
    for name in allbits:
        res["photon_objects_with_bit"][name] = 0
        res["events_fired_mass90_with_photon_bit"][name] = 0
    main = "HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90"
    nread = 0
    for arrays in t.iterate(branches, step_size="100 MB", entry_stop=max_events):
        n = len(arrays["TrigObj_id"])
        nread += n
        ids = arrays["TrigObj_id"]
        bits = ak.values_astype(arrays["TrigObj_filterBits"], np.uint64)
        flat_ids = ak.to_numpy(ak.flatten(ids))
        u, c = np.unique(flat_ids, return_counts=True)
        for i, k in zip(u.tolist(), c.tolist()):
            res["objects_per_id"][str(i)] = res["objects_per_id"].get(str(i), 0) + k
        for p in paths:
            res["fired"][p] += int(ak.sum(arrays[p]))
        isph = ids == 22
        fired_main = arrays[main] if main in keys else ak.zeros_like(arrays["TrigObj_id"][:, 0:0], dtype=bool)
        for name, bit in allbits.items():
            if bit is None:
                continue
            sel = isph & has_bit(bits, bit)
            res["photon_objects_with_bit"][name] += int(ak.sum(sel))
            if main in keys:
                res["events_fired_mass90_with_photon_bit"][name] += int(ak.sum(ak.any(sel, axis=1) & arrays[main]))
        if bit_ele_unseeded is not None:
            res["electron_objects_with_unseeded_bit"] += int(ak.sum((ids == 11) & has_bit(bits, bit_ele_unseeded)))
    res["events"] = nread
    res["seconds"] = round(time.time() - t0, 1)
    # verdict
    uns = [v for k, v in res["photon_objects_with_bit"].items() if k.startswith("unseeded:") and allbits[k] is not None]
    sed = [v for k, v in res["photon_objects_with_bit"].items() if k.startswith("seeded:") and allbits[k] is not None]
    if not any(b is not None for b in bits_unseeded.values()):
```

Backup code: verify_nanoaod_files.py (5/5)

scripts/verify_nanoaod_files.py, lines 130 to 163

```
        res["verdict"] = "diphoton unseeded bits not documented in this file (NanoAOD v11 or older): not affected"
    elif sum(sed) > 0 and sum(uns) == 0:
        res["verdict"] = "BUG REPRODUCED: seeded last-filter bits set %d times, unseeded last-filter bits never set" % sum(sed)
    elif sum(uns) > 0:
        res["verdict"] = "UNEXPECTED: unseeded bits set %d times (fixed file, or claim wrong: report this)" % sum(uns)
    else:
        res["verdict"] = "INCONCLUSIVE: no object with the seeded diphoton bits either (too few diphoton-triggered events?)"
    return res


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--label", required=True, help="NanoAOD version label, e.g. v13")
    ap.add_argument("--files", nargs="+", required=True)
    ap.add_argument("--max-events", type=int, default=300000)
    ap.add_argument("--out", default=None)
    a = ap.parse_args()
    results = []
    for p in a.files:
        print("==", a.label, p, flush=True)
        try:
            r = check_file(p, a.max_events)
        except Exception as e:  # keep going, record the failure
            r = {"file": p, "error": repr(e)}
        results.append(r)
        print(json.dumps({k: v for k, v in r.items() if k not in ("file",)}, indent=1), flush=True)
    out = a.out or "results_%s.json" % a.label
    json.dump({"label": a.label, "results": results}, open(out, "w"), indent=1)
    print("wrote", out)


if __name__ == "__main__":
    main()
```

Backup code: deadbits.py (1/8)

scripts/deadbits.py, lines 1 to 30

```
"""Resolve every trigger-object filter bit of a NanoAOD triggerObjects_cff.py against HLT menus.
usage: deadbits.py <triggerObjects_cff.py> <label=menu.py> [<label=menu.py> ...]
Status per bit and menu: OK (a matching filter whose objects the selection's coll() accepts),
COLL (matching filters exist but none passes the selection's coll()), STALE (no module matches),
UNK (matching filter, collection could not be determined)."""
import re, sys, ast

# ---------- menu ----------
def parse_modules(text):
    mods = {}
    for m in re.finditer(r'^(?:fragment|process)\.(\w+) = cms\.(\w+)\(', text, re.M):
        k = m.end(); depth = 1; q = None
        while k < len(text) and depth:
            c = text[k]
            if q:
                if c == q: q = None
            elif c in "\"'": q = c
            elif c == "(": depth += 1
            elif c == ")": depth -= 1
            k += 1
        mods[m.group(1)] = (m.group(2), text[m.end():k-1])
    return mods

CAND_PARAMS = ["l1EGCand", "inputTag", "CandTag", "inputCandCollection", "candIsolatedTag",
               "candNonIsolatedTag", "inputTag1", "inputTag2", "firstLegLastFilter", "secondLegLastFilter",
               "JetSrc", "src", "htLabels", "mhtLabels", "inputJetTag", "originalTag", "TauSrc", "jetSrc",
               "Jets", "JetTags", "inputJets", "candTag", "PreviousCandTag", "previousCandTag", "metLabels",
               "mhtLabel", "inputMet", "L1TauTrigger", "TriggerSrc", "JetTag", "MuonTag", "ElectronTag",
               "PhotonTag", "muonTag", "electronTag", "photonTag", "tauTag", "jetTag", "taus", "originTag1", "originTag2", "originTag3", "originTag", "jets"]
```

Backup code: deadbits.py (2/8)

scripts/deadbits.py, lines 31 to 58

```
def tag_labels(body, name):
    m = re.search(r'\b%s = cms\.(VInputTag|InputTag)\(\s*(.*?)\s*\)' % re.escape(name), body, re.S)
    if not m: return None
    kind, inner = m.group(1), m.group(2)
    toks = [t.strip().strip('"\'') for t in inner.replace("\n", " ").split(",") if t.strip()]
    if kind == "InputTag":
        return [toks[0].split(":")[0]] if toks else []
    return [t.split(":")[0] for t in toks]

def collections_of(mods, name, depth=0, seen=None):
    """Collections (EDProducer labels) whose objects a filter saves."""
    if seen is None: seen = set()
    if name in seen or depth > 8 or name not in mods: return set()
    seen.add(name)
    typ, body = mods[name]
    direct, follow = set(), []
    for p in CAND_PARAMS:
        labs = tag_labels(body, p)
        if not labs: continue
        for lab in labs:
            t2 = mods.get(lab, ("?", ""))[0]
            if t2 == "EDProducer": direct.add(lab)
            elif t2 == "EDFilter": follow.append(lab)
    if direct: return direct
    out = set()
    for f in follow: out |= collections_of(mods, f, depth + 1, seen)
    return out
```

Backup code: deadbits.py (3/8)

scripts/deadbits.py, lines 59 to 90

```
def glob_re(pat):
    parts = pat.split("*")
    return re.compile("^" + ".*".join(re.escape(p) for p in parts) + "$")

# ---------- config ----------
def block(text, name):
    lines = text.splitlines(); start = None
    for i, l in enumerate(lines):
        if re.match(r'^\s*%s\s*=\s*cms\.PSet\(' % name, l): start = i; break
    if start is None: return None
    indent = len(lines[start]) - len(lines[start].lstrip()); out = [lines[start]]
    for l in lines[start+1:]:
        ind = len(l) - len(l.lstrip())
        if l.strip() and ind ⩽ indent: break
        out.append(l)
    return "\n".join(out)

def split_args(s):
    args, depth, q, cur = [], 0, None, ""
    for c in s:
        if q:
            cur += c
            if c == q: q = None
            continue
        if c in "\"'": q = c; cur += c; continue
        if c in "([{": depth += 1
        if c in ")]}": depth -= 1
        if c == "," and depth == 0: args.append(cur); cur = ""; continue
        cur += c
    if cur.strip(): args.append(cur)
    return [a.strip() for a in args]
```

Backup code: deadbits.py (4/8)

scripts/deadbits.py, lines 91 to 124

```
def calls(blk, fname="mksel("):
    res, i = [], 0
    while True:
        j = blk.find(fname, i)
        if j < 0: break
        k = j + len(fname); depth = 1; q = None
        while k < len(blk) and depth:
            c = blk[k]
            if q:
                if c == q: q = None
            elif c in "\"'": q = c
            elif c == "(": depth += 1
            elif c == ")": depth -= 1
            k += 1
        res.append(blk[j+len(fname):k-1]); i = k
    return res

def chaintoken(tokens, op):
    if not isinstance(tokens, (list, tuple)):
        tokens = [t.strip() for t in tokens.split(op)]
    return tokens

def alternatives(sel0):
    """Emulate mksel: return list of alternatives; each alternative = list of (kind, pattern) all required."""
    s = sel0
    if isinstance(s, str) and "OR" in s: s = chaintoken(s, "OR")
    if isinstance(s, str) and "AND" in s: s = ("AND",) + tuple(chaintoken(s, "AND"))
    if isinstance(s, tuple) and s and s[0] == "AND": return [[("filter", t) for t in s[1:]]]
    if isinstance(s, list): return [[("filter", t)] for t in s]
    if isinstance(s, tuple): return [[("filter", t) for t in s]]
    toks = re.findall(r"(filter|path|coll)\('([^']*)'\)", s)
    if not toks: return [[("raw", s)]]
    if "min(" in s or "&&" in s:   # treat as AND of all tokens (approximation)
        return [toks]
```

Backup code: deadbits.py (5/8)

scripts/deadbits.py, lines 125 to 158

```
    return [[t] for t in toks]

def parse_config(text):
    sels = {}
    for name in ("Electron", "Photon", "Muon", "Tau", "BoostedTau", "Jet", "FatJet", "MET", "HT", "MHT"):
        blk = block(text, name)
        if blk is None: continue
        sel = re.search(r'sel\s*=\s*cms\.string\((["\'])(.*?)\1\)', blk)
        skip = re.search(r'skipObjectsNotPassingQualityBits\s*=\s*cms\.bool\((\w+)\)', blk)
        colls = re.findall(r"coll\('([^']*)'\)", sel.group(2)) if sel else []
        if sel and re.search(r"coll\('[^']*'\)\s*\|\|\s*filter\(", sel.group(2)): colls = []  # coll OR-ed with filters: not restrictive
        bits = []
        qb = blk[blk.find("qualityBits"):] if "qualityBits" in blk else ""
        if "cms.VPSet" in qb:
            for n, c in enumerate(calls(qb)):
                args = split_args(c)
                try: a0 = ast.literal_eval(args[0])
                except Exception: a0 = args[0].strip('"')
                doc = ""
                if len(args) > 1 and not args[1].startswith("bit"):
                    try: doc = ast.literal_eval(args[1])
                    except Exception: doc = args[1]
                b = re.search(r'bit\s*=\s*(\d+)', c)
                bits.append((int(b.group(1)) if b else n, a0, doc))
        else:  # old string layout: "a*filter('x') + ..."
            m = re.search(r'qualityBits\s*=\s*cms\.string\(\s*(.*?)\)\s*,\s*\n', blk, re.S)
            if m:
                expr = "".join(ast.literal_eval(p) for p in re.findall(r'"(?:[^"\\]|\\.)*"', m.group(1)))
                for term in expr.split("+"):
                    w = re.match(r'\s*(\d+)\*', term)
                    bitno = int(w.group(1)).bit_length() - 1 if w else 0
                    pats = re.findall(r"filter\('([^']*)'\)", term)
                    bits.append((bitno, pats if len(pats) > 1 else (pats[0] if pats else term.strip()), ""))
        sels[name] = dict(sel=sel.group(2) if sel else "", skip=skip.group(1) if skip else "?", colls=colls, bits=bits)
```

Backup code: deadbits.py (6/8)

scripts/deadbits.py, lines 159 to 192

```
    return sels

# ---------- evaluation ----------
def accepts(colls, label):
    if not colls: return True
    return any(glob_re(c).match(label) for c in colls)

def token_status(mods, filters, paths, producers, kind, pat, colls):
    if kind == "path":
        return "OK" if any(glob_re(pat).match(p) for p in paths) else "STALE"
    if kind == "coll":
        return "OK" if any(glob_re(pat).match(p) for p in producers) else "STALE"
    if kind == "raw":
        return "UNK"
    rx = glob_re(pat)
    matched = [f for f in filters if rx.match(f)]
    if not matched: return "STALE"
    best, info = "COLL", []
    for f in matched:
        cs = collections_of(mods, f)
        if not cs: best = "UNK" if best != "OK" else best; info.append("%s[%s:?]" % (f, mods[f][0])); continue
        if any(accepts(colls, c) for c in cs): return "OK"
        info.append("%s[%s:%s]" % (f, mods[f][0], ",".join(sorted(cs))))
    return best + "{" + "; ".join(info[:3]) + "}"

RANK = {"OK": 0, "UNK": 1, "COLL": 2, "STALE": 3}
def bit_status(mods, filters, paths, producers, alts, colls):
    results = []
    for alt in alts:
        sts = [token_status(mods, filters, paths, producers, k, p, colls) for k, p in alt]
        worst = max(sts, key=lambda s: RANK[s.split("{")[0]])
        results.append(worst)
    return min(results, key=lambda s: RANK[s.split("{")[0]])
```

Backup code: deadbits.py (7/8)

scripts/deadbits.py, lines 193 to 224

```
def main():
    cfg = open(sys.argv[1]).read()
    sels = parse_config(cfg)
    menus = []
    for arg in sys.argv[2:]:
        label, path = arg.split("=", 1)
        mods = parse_modules(open(path).read())
        filters = [n for n, (t, b) in mods.items() if t == "EDFilter"]
        paths = [n for n, (t, b) in mods.items() if t == "Path"]
        producers = [n for n, (t, b) in mods.items() if t == "EDProducer"]
        menus.append((label, mods, filters, paths, producers))
    print("config:", sys.argv[1]); print("menus:", " ".join(l for l, *_ in menus))
    for name, s in sels.items():
        print("\n== %s (sel colls: %s; skip=%s; %d bits)" % (name, s["colls"] or "any", s["skip"], len(s["bits"])))
        if not s["bits"] and s["skip"] == "True":
            print("   !! no quality bits and skipObjectsNotPassingQualityBits=True: this object type is never written")
        counts = {l: {"OK": 0, "UNK": 0, "COLL": 0, "STALE": 0} for l, *_ in menus}
        for bitno, a0, doc in s["bits"]:
            alts = alternatives(a0)
            row = []
            for label, mods, filters, paths, producers in menus:
                st = bit_status(mods, filters, paths, producers, alts, s["colls"])
                counts[label][st.split("{")[0]] += 1
                row.append((label, st))
            if any(st.split("{")[0] != "OK" for l, st in row):
                desc = a0 if isinstance(a0, str) else "|".join(a0)
                print("   bit %2d %-70s %s" % (bitno, (desc[:68] + (".." if len(desc) > 68 else "")), " ".join("%s:%s" % (l, st.split("{")[0]) for l, st in row)))
                extra = [st for l, st in row if "{" in st]
                if extra: print("         detail: " + extra[0][extra[0].find("{"):][:260])
        print("   summary: " + " | ".join("%s OK=%d COLL=%d STALE=%d UNK=%d" % (l, c["OK"], c["COLL"], c["STALE"], c["UNK"]) for l, c in counts.items()))

main()
```

Backup code: deadbits.py (8/8)

scripts/deadbits.py, lines 225 to 225

Backup code: menuscan.py (1/3)

scripts/menuscan.py, lines 1 to 33

```
import re, sys

def parse_modules(text):
    mods = {}
    pat = re.compile(r'^(?:fragment|process)\.(\w+) = cms\.(\w+)\(', re.M)
    for m in pat.finditer(text):
        k = m.end(); depth = 1; q = None
        while k < len(text) and depth:
            c = text[k]
            if q:
                if c == q: q = None
            elif c in "\"'": q = c
            elif c == "(": depth += 1
            elif c == ")": depth -= 1
            k += 1
        mods[m.group(1)] = (m.group(2), text[m.end():k-1])
    return mods

def param(body, name):
    m = re.search(r'\b%s = cms\.\w+\(\s*(.*?)\s*\)' % re.escape(name), body, re.S)
    return m.group(1).replace('"', '').replace("'", "").replace("\n", " ") if m else None

def members(body):
    # sequence/path members in order
    return re.findall(r'(?:fragment|process)\.(\w+)', body)

def describe(mods, name):
    if name not in mods: return "%s: NOT DEFINED" % name
    typ, body = mods[name]
    edtype = re.match(r'\s*"(\w+)"', body)
    edtype = edtype.group(1) if edtype else "?"
    bits = []
    for p in ("saveTags", "l1EGCand", "inputTag", "candTag", "triggerType", "varTag", "lessThan", "useEt", "thrRegularEB", "thrRegularEE", "thrOverEEB", "thrOverEEE",
"thrOverE2EB", "thrOverE2EE", "ncandcut", "etcutEB", "etcutEE", "lowerMassCut", "upperMassCut", "minPt", "minMass", "maxMass"):
```

Backup code: menuscan.py (2/3)

scripts/menuscan.py, lines 34 to 65

```
        v = param(body, p)
        if v is not None: bits.append("%s=%s" % (p, v))
    return "%s [%s] %s" % (name, edtype, " ".join(bits))

def main(path, pathre, maxdepth=2):
    text = open(path).read()
    m = re.search(r'tableName = cms\.string\(\s*"([^"]+)"', text)
    print("#" * 110); print(path, "menu:", m.group(1) if m else "?")
    mods = parse_modules(text)
    names = sorted(n for n, (t, b) in mods.items() if t == "Path" and re.search(pathre, n))
    print("paths:", ", ".join(names))
    for n in names:
        if not re.search(r'Diphoton30_(22|18)', n): continue
        print("-" * 100); print("PATH", n)
        seen = set()
        def walk(name, depth):
            typ, body = mods.get(name, ("?", ""))
            for mem in members(body):
                if mem in seen: continue
                seen.add(mem)
                t2 = mods.get(mem, ("?", ""))[0]
                if t2 == "Sequence" and depth < maxdepth and not mem.startswith(("HLTBeginSequence", "HLTEndSequence", "HLTDoLocalHcal", "HLTDoFull", "HLTPFClustering",
"HLTEgamma", "HLTL1", "HLTDoLocalPixel", "HLTRecoPixel", "HLTElePixel", "HLTTrackReco", "HLTPFTrack", "HLTPFRecoSequence", "HLTDo")):
                    print("  " * depth + "SEQ " + mem)
                    walk(mem, depth + 1)
                elif t2 == "EDFilter":
                    print("  " * depth + describe(mods, mem))
                elif t2 == "Sequence":
                    print("  " * depth + "seq " + mem + " (not expanded)")
        walk(n, 1)

if __name__ == "__main__":
    main(sys.argv[1], sys.argv[2] if len(sys.argv) > 2 else r'HLT_(Diphoton|DoublePhoton)')
```

Backup code: menuscan.py (3/3)

scripts/menuscan.py, lines 66 to 66

Backup code: menusummary.py (1/3)

scripts/menusummary.py, lines 1 to 34

```
import re, sys, hashlib

def parse_modules(text):
    mods = {}
    pat = re.compile(r'^(?:fragment|process)\.(\w+) = cms\.(\w+)\(', re.M)
    for m in pat.finditer(text):
        k = m.end(); depth = 1; q = None
        while k < len(text) and depth:
            c = text[k]
            if q:
                if c == q: q = None
            elif c in "\"'": q = c
            elif c == "(": depth += 1
            elif c == ")": depth -= 1
            k += 1
        mods[m.group(1)] = (m.group(2), text[m.end():k-1])
    return mods

def param(body, name):
    m = re.search(r'\b%s = cms\.\w+\(\s*(.*?)\s*\)' % re.escape(name), body, re.S)
    return m.group(1).replace('"', '').replace("'", "").replace("\n", " ") if m else None

def members(body):
    return re.findall(r'(?:fragment|process)\.(\w+)', body)

KEY = ["hltDiEG22EtEta2p55UnseededFilter",
       "hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter",
       "hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter",
       "hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter",
       "hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter",
       "hltEG30Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eEcalIsoFilter",
       "hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter",
       "hltEG30Iso60b40eCaloId11b32eHE10b9eR9Id50b90eEcalIsoFilter",
       "hltDiEG18TrackIso60Iso60b40eCaloId11b32eHE10b9eR9Id50b90eTrackIsoUnseededFilter",
```

Backup code: menusummary.py (2/3)

scripts/menusummary.py, lines 35 to 66

```
       "hltDiEG18EtUnseededFilter"]

def leaves(mods, name, seen=None):
    if seen is None: seen = set()
    out = []
    typ, body = mods.get(name, ("?", ""))
    for mem in members(body):
        if mem in seen: continue
        seen.add(mem)
        t2 = mods.get(mem, ("?", ""))[0]
        if t2 in ("Sequence", "Path"):
            out += leaves(mods, mem, seen)
        elif t2 == "EDFilter":
            out.append(mem)
    return out

for path in sys.argv[1:]:
    text = open(path).read()
    m = re.search(r'tableName = cms\.string\(\s*["\']([^"\']+)["\']', text)
    mods = parse_modules(text)
    print("#" * 100)
    print("%s\n  menu: %s" % (path.split("/")[-1], m.group(1) if m else "?"))
    names = sorted(n for n, (t, b) in mods.items() if t == "Path" and re.search(r'HLT_(Diphoton|DoublePhoton)', n))
    print("  paths: " + ", ".join(n.replace("HLT_", "") for n in names))
    for k in KEY:
        if k not in mods:
            print("  KEY %-88s : not defined" % k); continue
        typ, body = mods[k]
        ed = re.match(r'\s*"(\w+)"', body).group(1)
        print("  KEY %-88s : %s saveTags=%s l1EGCand=%s inputTag=%s triggerType=%s" % (k, ed, param(body, "saveTags"), param(body, "l1EGCand"), param(body, "inputTag"),
param(body, "triggerType")))
    for n in names:
        if not re.search(r'Diphoton30_(22|18)', n): continue
```

Backup code: menusummary.py (3/3)

scripts/menusummary.py, lines 67 to 78

```
        lv = [f for f in leaves(mods, n) if f.startswith("hlt") and not f.startswith("hltPre") and mods[f][1].find("HLTL1TSeed") < 0 and not
re.search(r'(Trigger|Bool|HLTPrescaler)', mods[f][1][:40])]
        egf = [f for f in lv if re.search(r'EG|Diphoton|DiEG|DiPhoton|Mass|Photon', f)]
        uns = [f for f in egf if "Unseeded" in f]
        sed = [f for f in egf if "Unseeded" not in f and not re.search(r'Comb|Mass', f)]
        comb = [f for f in egf if re.search(r'Comb|Mass', f)]
        st = set(param(mods[f][1], "saveTags") for f in egf)
        h = hashlib.md5(" ".join(egf).encode()).hexdigest()[:8]
        print("  PATH %s: nEgFilters=%d chain=%s saveTags=%s" % (n.replace("HLT_", ""), len(egf), h, "/".join(sorted(x or "None" for x in st))))
        print("       last seeded   : %s" % (sed[-1] if sed else "-"))
        print("       last unseeded : %s" % (uns[-1] if uns else "-"))
        print("       comb/mass     : %s" % ", ".join(comb))
```

Backup code: pathmatrix.py (1/1)

scripts/pathmatrix.py, lines 1 to 22

```
import re, sys
def paths(path):
    text = open(path).read()
    m = re.search(r'tableName = cms\.string\(\s*["\']([^"\']+)["\']', text)
    res = {}
    for pm in re.finditer(r'^(?:fragment|process)\.(HLT_(?:Diphoton|DoublePhoton|DiphotonMVA)\w*?)_v(\d+) = cms\.Path\(', text, re.M):
        res[pm.group(1)] = int(pm.group(2))
    return (m.group(1) if m else "?"), res
cols = []
allp = set()
for f in sys.argv[1:]:
    name, res = paths(f)
    cols.append((f.split("/")[-1].replace("menu_", "").replace(".py", ""), name, res))
    allp |= set(res)
print("columns:")
for i, (f, n, r) in enumerate(cols):
    print("  c%-2d %-32s %s" % (i, f, n))
print()
print("%-78s %s" % ("path", " ".join("c%-2d" % i for i in range(len(cols)))))
for p in sorted(allp):
    print("%-78s %s" % (p.replace("HLT_", ""), " ".join("%-3s" % (r.get(p, "-")) for f, n, r in cols)))
```

Backup code: bits.py (1/2)

scripts/bits.py, lines 1 to 34

```
import re, sys

def block(text, name):
    lines = text.splitlines()
    start = None
    for i, l in enumerate(lines):
        if re.match(r'^\s*%s\s*=\s*cms\.PSet\(' % name, l):
            start = i; break
    if start is None:
        return None
    indent = len(lines[start]) - len(lines[start].lstrip())
    out = [lines[start]]
    for l in lines[start+1:]:
        ind = len(l) - len(l.lstrip())
        if l.strip() and ind <= indent:
            break
        out.append(l)
    return "\n".join(out)

def calls(blk, fname="mksel("):
    res = []
    i = 0
    while True:
        j = blk.find(fname, i)
        if j < 0: break
        k = j + len(fname); depth = 1; q = None
        while k < len(blk) and depth:
            c = blk[k]
            if q:
                if c == q: q = None
            elif c in "\"'": q = c
            elif c == "(": depth += 1
            elif c == ")": depth -= 1
            k += 1
```

Backup code: bits.py (2/2)

scripts/bits.py, lines 35 to 62

```
        res.append(blk[j+len(fname):k-1])
        i = k
    return res

for path in sys.argv[1:]:
    text = open(path).read()
    print("=" * 100); print(path)
    for name in ("Photon", "Electron"):
        blk = block(text, name)
        if blk is None:
            print("  %s: block not found" % name); continue
        sel = re.search(r'sel\s*=\s*cms\.string\((["\'])(.*?)\1\)', blk)
        print("  %s sel: %s" % (name, sel.group(2) if sel else "?"))
        skip = re.search(r'skipObjectsNotPassingQualityBits\s*=\s*cms\.bool\((\w+)\)', blk)
        print("  %s skip: %s" % (name, skip.group(1) if skip else "?"))
        if name == "Electron":
            continue
        cs = calls(blk)
        if not cs:
            qb = re.search(r'qualityBits\s*=\s*cms\.string\((["\'])(.*?)\1', blk, re.S)
            print("  old layout qualityBits string:", qb.group(2)[:2000] if qb else "none")
            continue
        for n, c in enumerate(cs):
            b = re.search(r'bit\s*=\s*(\d+)', c)
            flag = ""
            if "Unseeded" in c: flag = "  ⟵ unseeded"
            print("   %2d%s %s%s" % (n, (" (bit=%s)" % b.group(1)) if b else "", c.strip()[:150], flag))
```

Backup code: bitfilters.py (1/2)

scripts/bitfilters.py, lines 1 to 29

```
import re, sys
def parse_modules(text):
    mods = {}
    for m in re.finditer(r'^(?:fragment|process)\.(\w+) = cms\.(\w+)\(', text, re.M):
        k = m.end(); depth = 1; q = None
        while k < len(text) and depth:
            c = text[k]
            if q:
                if c == q: q = None
            elif c in "\"'": q = c
            elif c == "(": depth += 1
            elif c == ")": depth -= 1
            k += 1
        mods[m.group(1)] = (m.group(2), text[m.end():k-1])
    return mods
def param(body, name):
    m = re.search(r'\b%s = cms\.\w+\(\s*(.*?)\s*\)' % re.escape(name), body, re.S)
    return m.group(1).replace('"', '').replace("'", "").strip() if m else None
FILTERS =
["hltEG33L1EG26HEFilter","hltEG50HEFilter","hltEG75HEFilter","hltEG90HEFilter","hltEG120HEFilter","hltEG150HEFilter","hltEG175HEFilter","hltEG200HEFilter","hltHtEcal800","hl
tEG45EBTightIDTightIsoTrackIsoFilter","hltEG50EBTightIDTightIsoTrackIsoFilter","hltEG110EBTightIDTightIsoTrackIsoFilter","hltEG120EBTightIDTightIsoTrackIsoFilter","hltMu17Ph
oton30IsoCaloIdPhotonlegTrackIsoFilter","hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter","hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter","hltEG22Iso60CaloId15b35e
HE12R9Id50b80eTrackIsoUnseededLastFilter","hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter","hltEG30Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eEcalIsoFilter","hltEG18T
rackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter","hltEG22L1VBFLooseIsoEGHEFilter"]
menus = []
for f in sys.argv[1:]:
    text = open(f).read()
    mods = parse_modules(text)
    row = {}
    for name in FILTERS:
        if name not in mods: row[name] = "-"; continue
        typ, body = mods[name]
        coll = param(body, "l1EGCand") or param(body, "inputTag") or "?"
        if "," in coll: coll = coll.split(",")[0]
```

Backup code: bitfilters.py (2/2)

scripts/bitfilters.py, lines 30 to 36

```
        row[name] = {"hltEgammaCandidates": "S", "hltEgammaCandidatesUnseeded": "U"}.get(coll, coll)
    menus.append((f.split("/")[-1].replace("menu_","").replace(".py",""), row))
print("%-82s %s" % ("filter", " ".join("%-3s" % ("c%d" % i) for i in range(len(menus)))))
for i, (n, r) in enumerate(menus): print("  c%-2d %s" % (i, n))
for name in FILTERS:
    print("%-82s %s" % (name, " ".join("%-3s" % r[name] for n, r in menus)))
```

Backup code: wfn.py (1/1)

scripts/wfn.py, lines 1 to 27

```
import re, sys
class WFN:
    def __init__(self, offset):
        self.offset = offset; self.index = 1; self.thousands = 0; self.hundreds = 0
    def __call__(self):
        v = float(f"{self.offset}.{self.thousands}{self.hundreds}{self.index:02d}"); self.index += 1; return v
    def subnext(self):
        self.hundreds += 1; self.index = 1
    def next(self, index=None):
        if index is None: self.thousands += 1; self.hundreds = 0
        else: self.thousands = index; self.hundreds = 0
        self.index = 1
src = open(sys.argv[1]).read()
m = re.search(r'_wfn\s*=\s*WFN\((\d+)\)', src)
w = WFN(int(m.group(1))) if m else None
print("counter init:", m.group(0) if m else "none (old scheme?)")
for i, line in enumerate(src.splitlines(), 1):
    s = line.strip()
    if s.startswith("#"): continue
    if re.match(r'_wfn\.next\(\s*\)', s): w.next()
    elif re.match(r'_wfn\.next\((\d+)\)', s): w.next(int(re.match(r'_wfn\.next\((\d+)\)', s).group(1)))
    elif re.match(r'_wfn\.subnext\(\)', s): w.subnext()
    elif "workflows[_wfn()]" in s:
        print("%-10s %s" % (w(), s.split("=", 1)[1].strip()[:110]))
    elif re.match(r'workflows\[([\d.]+)\]', s):
        print("%-10s %s" % (re.match(r'workflows\[([\d.]+)\]', s).group(1), s.split("=", 1)[1].strip()[:110]))
```

Backup code: measure_trigobj.py (1/7)

scripts/measure_trigobj.py, lines 1 to 33

```
#!/usr/bin/env python3
"""Count TrigObj filter bits of the Diphoton30_22 photon legs in one NanoAOD file.

Bit numbers are parsed from the TrigObj_filterBits branch documentation, never hardcoded.
Usage: python3 measure_trigobj.py <version> <dataset> <file> <out.json> [<reference file>]
With a reference file, events are matched by (run, lumi, event) and their TrigObj content compared.
"""
import json
import re
import sys

import awkward as ak
import numpy as np
import uproot

MAX_EVENTS = 300000
PATH_MASS90 = "HLT_Diphoton30_22_R9Id_OR_IsoCaloId_AND_HE_R9Id_Mass90"
PATH_DIELE = "HLT_DoubleEle33_CaloIdL_MW"
PHOTON_FILTERS = {
    "R9UnseededLast": "hltEG22R9Id85b90eHE12R9Id50b80eR9UnseededLastFilter",
    "TrackIsoUnseededLast": "hltEG22Iso60CaloId15b35eHE12R9Id50b80eTrackIsoUnseededLastFilter",
    "EG18Unseeded": "hltEG18TrackIso60Iso60CaloId15b35eR9Id50b90eHE12b10eR9Id50b80eTrackIsoUnseededFilter",
    "seeded_R9IdLast": "hltEG30LR9Id85b90eHE12R9Id50b80eR9IdLastFilter",
    "seeded_EcalIsoLast": "hltEG30LIso60CaloId15b35eHE12R9Id50b80eEcalIsoLastFilter",
    "hltHtEcal800": "hltHtEcal800",
}
SEEDED = ("seeded_R9IdLast", "seeded_EcalIsoLast")
UNSEEDED = ("R9UnseededLast", "TrackIsoUnseededLast", "EG18Unseeded")
EXPECTED_PHOTON_BIT_COUNTS = (0, 12, 18, 21)
BIT_ENTRY = re.compile(r"(\d+)\s*⇒\s*(.*?)(?=,\s*\d+\s*⇒|\s+for\s+\w+\s*$)")
EVENT_COLUMNS = ["run", "luminosityBlock", "event", "TrigObj_id", "TrigObj_filterBits", "TrigObj_pt"]
```

Backup code: measure_trigobj.py (2/7)

scripts/measure_trigobj.py, lines 34 to 67

```
def bits_of(doc, obj):
    # Total: time O(len(doc)), space O(n_bits)
    bits = {}
    for chunk in doc.split("; "):                                 # list scan O(len) | each per-object chunk
        if not chunk.strip().endswith(" for " + obj):             # -                | keep only this object's chunk
            continue
        for match in BIT_ENTRY.finditer(chunk):                   # regex O(len)     | each "N ⇒ description"
            bits[int(match.group(1))] = match.group(2).strip()  # dict insert O(1) | bit → description
    return bits


def find_bit(bits, name):
    # Total: time O(n_bits), space O(1)
    for bit, description in bits.items():                         # dict scan O(n)   | first bit naming the filter
        if name in description:                                   # substring O(len) | description contains name
            return bit
    return None


def has_bit(filter_bits, bit):
    # Total: time O(n_obj), space O(n_obj)
    shifted = filter_bits >> np.uint64(bit)                       # jagged op O(n)   | move bit to position 0
    return (shifted & np.uint64(1)) == 1                          # jagged op O(n)   | per-object bit set?


def count_objects(ids, filter_bits, object_id, bit, event_mask=None):
    # Total: time O(n_obj), space O(n_obj)
    if bit is None:
        return None
    selected = (ids == object_id) & has_bit(filter_bits, bit)  # jagged mask O(n) | objects of this id with bit
    if event_mask is not None:
        selected = selected & event_mask                          # broadcast O(n)   | only in chosen events
    return int(ak.sum(selected))                                  # reduction O(n)   | number of objects
```

Backup code: measure_trigobj.py (3/7)

scripts/measure_trigobj.py, lines 68 to 94

```
def bit_counts_per_id(ids, filter_bits, n_bits, skip_id):
    # Total: time O(n_obj * n_bits), space O(n_obj)
    flat_ids = ak.to_numpy(ak.flatten(ids))                    # array O(n)       | all object ids
    flat_bits = ak.to_numpy(ak.flatten(filter_bits))           # array O(n)       | all object bit words
    counts = {}
    for object_id in np.unique(flat_ids):                      # sort O(n log n)  | each object type
        if object_id == skip_id:
            continue
        words = flat_bits[flat_ids == object_id]               # mask O(n)        | bit words of this type
        per_bit = [int(((words >> np.uint64(b)) & np.uint64(1)).sum()) for b in range(n_bits)]
        counts[str(int(object_id))] = {str(b): c for b, c in enumerate(per_bit) if c > 0}
    return counts


def event_contents(arrays, skip_id=None):
    # Total: time O(n_obj log n_obj), space O(n_obj)
    contents = {}
    rows = zip(arrays["run"], arrays["luminosityBlock"], arrays["event"],
               ak.to_list(arrays["TrigObj_id"]), ak.to_list(arrays["TrigObj_filterBits"]),
               ak.to_list(arrays["TrigObj_pt"]))
    for run, lumi, event, ids, words, pts in rows:             # loop O(n_events) | one event at a time
        objects = [(i, int(w) & 0xFFFFFFFFFFFFFFFF, p) for i, w, p in zip(ids, words, pts) if i ≠ skip_id]
        contents[(int(run), int(lumi), int(event))] = sorted(objects)  # dict O(1)  | key → sorted objects
    return contents
```

Backup code: measure_trigobj.py (4/7)

scripts/measure_trigobj.py, lines 95 to 128

```
def compare_with_reference(arrays, reference_url):
    # Total: time O(n_obj log n_obj), space O(n_obj)
    reference = uproot.open(reference_url)["Events"].arrays(EVENT_COLUMNS, entry_stop=MAX_EVENTS)
    this_all, ref_all = event_contents(arrays), event_contents(reference)
    this_other, ref_other = event_contents(arrays, skip_id=22), event_contents(reference, skip_id=22)
    common = set(this_all) & set(ref_all)                      # set join O(n)    | events in both files
    identical_all = sum(1 for k in common if this_all[k] == ref_all[k])
    identical_other = sum(1 for k in common if this_other[k] == ref_other[k])
    return {
        "reference_file": reference_url,
        "events_in_reference": len(ref_all),
        "events_in_common": len(common),
        "events_identical_all_TrigObj": identical_all,
        "events_identical_TrigObj_without_id22": identical_other,
    }


def main():
    # Total: time O(n_events * n_obj), space O(n_events * n_obj)
    version, dataset, url, out_path = sys.argv[1:5]
    reference_url = sys.argv[5] if len(sys.argv) > 5 else None
    root_file = uproot.open(url)
    tree = root_file["Events"]
    branch = tree["TrigObj_filterBits"]
    doc = branch.title
    photon_bits = bits_of(doc, "Photon")                       # dict             | photon bit → name
    electron_bits = bits_of(doc, "Electron")                   # dict             | electron bit → name
    bit = {key: find_bit(photon_bits, name) for key, name in PHOTON_FILTERS.items()}
    electron_bit = find_bit(electron_bits, "CaloIdL_MW unseeded")
    n_bits = 64 if "u8" in str(branch.interpretation) else 32

    paths = [p for p in (PATH_MASS90, PATH_DIELE) if p in tree.keys()]
    arrays = tree.arrays(EVENT_COLUMNS + paths, entry_stop=MAX_EVENTS)  # columnar read | up to 300k events
    ids = arrays["TrigObj_id"]
```

Backup code: measure_trigobj.py (5/7)

scripts/measure_trigobj.py, lines 129 to 162

```
    filter_bits = ak.values_astype(arrays["TrigObj_filterBits"], np.uint64)
    n_events = len(ids)
    assert n_events > 0, "no events read"

    fired = arrays[PATH_MASS90] if PATH_MASS90 in paths else None
    n_fired = int(ak.sum(fired)) if fired is not None else None

    photons_with_bit = {key: count_objects(ids, filter_bits, 22, b) for key, b in bit.items()}
    photons_with_bit_fired = {key: count_objects(ids, filter_bits, 22, b, fired) if fired is not None else None
                              for key, b in bit.items()}

    unseeded_any = ak.zeros_like(ids, dtype=bool)              # jagged bool O(n) | photon has any unseeded bit
    for key in UNSEEDED:
        if bit[key] is not None:
            unseeded_any = unseeded_any | ((ids == 22) & has_bit(filter_bits, bit[key]))
    offending = ak.any(unseeded_any, axis=1)                   # reduction O(n)   | events with such a photon
    offending_events = [[int(r), int(l), int(e)] for r, l, e in zip(arrays["run"][offending][:50],
                        arrays["luminosityBlock"][offending][:50], arrays["event"][offending][:50])]

    unseeded_pt = ak.to_numpy(ak.flatten(arrays["TrigObj_pt"][unseeded_any]))  # array O(n) | pt of those objects
    photons_per_event = ak.sum(ids == 22, axis=1)              # reduction O(n)   | photon objects per event
    n_fired_with_unseeded = int(ak.sum(offending & fired)) if fired is not None else None

    flat_ids = ak.to_numpy(ak.flatten(ids))
    unique_ids, counts = np.unique(flat_ids, return_counts=True)  # sort O(n log n) | objects per id
    objects_per_id = {int(i): int(c) for i, c in zip(unique_ids, counts)}

    result = {
        "version": version,
        "dataset": dataset,
        "file": url,
        "file_size_bytes": getattr(root_file.file.source, "num_bytes", None),
        "events_in_file": int(tree.num_entries),
        "events_read": n_events,
```

Backup code: measure_trigobj.py (6/7)

scripts/measure_trigobj.py, lines 163 to 196

```
        "filterBits_interpretation": str(branch.interpretation),
        "n_photon_bits_documented": len(photon_bits),
        "n_electron_bits_documented": len(electron_bits),
        "photon_bit_numbers": bit,
        "electron_CaloIdL_MW_unseeded_bit": electron_bit,
        "paths_present": paths,
        "fired_Mass90": n_fired,
        "fired_DoubleEle33_CaloIdL_MW": int(ak.sum(arrays[PATH_DIELE])) if PATH_DIELE in paths else None,
        "photons_with_bit": photons_with_bit,
        "photons_with_bit_in_Mass90_events": photons_with_bit_fired,
        "electrons_with_CaloIdL_MW_unseeded_bit": count_objects(ids, filter_bits, 11, electron_bit),
        "objects_per_id": objects_per_id,
        "n_id2_MET": objects_per_id.get(2, 0),
        "n_id1515_BoostedTau": objects_per_id.get(1515, 0),
        "n_id22_Photon": objects_per_id.get(22, 0),
        "events_with_unseeded_photon_bit": int(ak.sum(offending)),
        "first_offending_run_lumi_event": offending_events,
        "Mass90_events_with_unseeded_photon_bit": n_fired_with_unseeded,
        "fraction_Mass90_events_with_unseeded_photon_bit":
            n_fired_with_unseeded / n_fired if n_fired else None,
        "photons_per_Mass90_event_mean": float(ak.mean(photons_per_event[fired])) if n_fired else None,
        "unseeded_photon_pt": {
            "n": int(len(unseeded_pt)),
            "min": float(unseeded_pt.min()) if len(unseeded_pt) else None,
            "median": float(np.median(unseeded_pt)) if len(unseeded_pt) else None,
            "n_below_22": int((unseeded_pt < 22).sum()),
        },
        "nTrigObj_mean": float(ak.mean(ak.num(ids))),
        "bit_counts_per_id_without_22": bit_counts_per_id(ids, filter_bits, n_bits, skip_id=22),
        "photon_bits_documented": {str(k): v for k, v in sorted(photon_bits.items())},
    }
    if reference_url:
        result["reference_comparison"] = compare_with_reference(arrays, reference_url)
```

Backup code: measure_trigobj.py (7/7)

scripts/measure_trigobj.py, lines 197 to 210

```
    if len(photon_bits) not in EXPECTED_PHOTON_BIT_COUNTS:
        print("WARNING: unexpected number of documented photon bits:", len(photon_bits))
    if result["events_with_unseeded_photon_bit"] > 0:
        print("NOTE: photon objects WITH an unseeded bit found:", result["events_with_unseeded_photon_bit"], "events")

    with open(out_path, "w") as out:
        json.dump(result, out, indent=1)
    skip = ("photon_bits_documented", "bit_counts_per_id_without_22")
    print(json.dumps({k: v for k, v in result.items() if k not in skip}, indent=1))


if __name__ == "__main__":
    main()
```

Backup code: make_fixed_nano.sh (1/3)

scripts/make_fixed_nano.sh, lines 1 to 33

```
#!/bin/bash
# Reproduce NanoAOD from one MiniAOD file twice, without and with the photon TrigObj fix.
# Usage: bash make_fixed_nano.sh v13|v15
set -eo pipefail

export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
source /cvmfs/cms.cern.ch/cmsset_default.sh

WORK=/eos/user/s/smahata/trigobj_check/cmssw
# parent MiniAOD of the central v13 file 4f45fe87-a958-4c8f-b88f-c322fec857f3.root
INPUT=root://cms-xrd-global.cern.ch//store/mc/Run3Summer23MiniAODv4/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/MINIAODSIM/130X_mcRun3_2023_realistic_v15_ext1-
v4/2820000/74338d47-0b38-4e0b-83a2-ad85c7f4e533.root
CFF=PhysicsTools/NanoAOD/python/triggerObjects_cff.py

case "$1" in
    v13)
        RELEASE=CMSSW_13_3_1_patch1
        ERA=Run3_2023
        GT=133X_mcRun3_2023_realistic_ForNanov13_v1
        ;;
    v15)
        RELEASE=CMSSW_15_0_15_patch4
        ERA=Run3_2023,run3_nanoAOD_pre142X
        GT=150X_mcRun3_2023_realistic_v1    # processing string of Run3Summer23NanoAODv15 in DAS
        ;;
    *)
        echo "usage: $0 v13|v15"; exit 1
        ;;
esac
OUT=$WORK/$1
mkdir -p "$OUT"

run_nano() {
    local label=$1
```

Backup code: make_fixed_nano.sh (2/3)

scripts/make_fixed_nano.sh, lines 34 to 66

```
    cmsDriver.py nano_$label --step NANO --mc --scenario pp --nThreads 4 \
        --eventcontent NANOAODSIM --datatier NANOAODSIM \
        --conditions $GT --era $ERA \
        --filein $INPUT --fileout file:$OUT/nano_$label.root \
        --python_filename $OUT/nano_${label}_cfg.py -n -1
}

echo "=== step 1: release area $RELEASE"
cd $WORK
if [ ! -d $RELEASE ]; then
    scram project CMSSW $RELEASE
fi
cd $RELEASE/src
eval $(scramv1 runtime -sh)

echo "=== step 2: check out PhysicsTools/NanoAOD and build"
if [ ! -d PhysicsTools/NanoAOD ]; then
    git cms-addpkg PhysicsTools/NanoAOD
fi
git checkout -- $CFF
scram b -j 8

echo "=== step 3: NANO without the fix"
grep -n "coll('hltEgammaCandidates')\"" $CFF
cd $OUT
run_nano nofix
cd $CMSSW_BASE/src

echo "=== step 4: apply the one-line fix"
sed -i "s#type(92) && pt > 15 && coll('hltEgammaCandidates')\"#type(92) \&\& pt > 15 \&\& (coll('hltEgammaCandidates') || coll('hltEgammaCandidatesUnseeded'))\"#" $CFF
git diff $CFF
CHANGED=$(git diff --numstat | awk '{print $1, $2}')
if [ "$CHANGED" != "1 1" ]; then
```

Backup code: make_fixed_nano.sh (3/3)

scripts/make_fixed_nano.sh, lines 67 to 76

```
    echo "ERROR: expected exactly one changed line, got: $CHANGED"; exit 1
fi
scram b -j 8

echo "═══ step 5: NANO with the fix"
cd $OUT
run_nano fix
ls -l $OUT/nano_nofix.root $OUT/nano_fix.root
echo "═══ done $1"
```

Backup code: relval_fix_check.sh (1/3)

scripts/relval_fix_check.sh, lines 1 to 34

```
#!/bin/bash
# Relval check of the photon TrigObj fix in CMSSW_16_1_3: clean area vs area with commit aebf3eb64.
# Steps 1-2 of each workflow run once (clean area); steps 3-4 run in both areas on the same step-2 output.
# Usage: bash relval_fix_check.sh
set -eo pipefail

export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
source /cvmfs/cms.cern.ch/cmsset_default.sh

RELEASE=CMSSW_16_1_3
WORK=/eos/user/s/smahata/trigobj_check/relval_16_1_3
FIX_COMMIT=aebf3eb64b9c525e77a99405f8a97aa254b335e9
CFF=PhysicsTools/NanoAOD/python/triggerObjects_cff.py
mkdir -p $WORK

echo "≡≡≡ step 1: clean area and full workflows"
cd $WORK
if [ ! -d clean ]; then
    scram project -n clean CMSSW $RELEASE
fi
cd clean/src
eval $(scramv1 runtime -sh)
mkdir -p $WORK/run_clean && cd $WORK/run_clean
# 12452.0 generates its own GEN-SIM: its RelVal GEN-SIM input is not found at T2_CH_CERN (--ibeos)
if ! ls 12452.0_*/step3_inNANOEDMAODSIM.root > /dev/null 2>&1; then
    rm -rf 12452.0_*
    runTheMatrix.py -w upgrade -l 12452.0 -j 4
fi
# 141.046 RAW is on tape only in DAS; the IB EOS cache (--ibeos) has the file
if ! ls 141.046_*/step3_inNANOEDMAOD.root > /dev/null 2>&1; then
    if ! ls -d 141.046_* > /dev/null 2>&1; then
        runTheMatrix.py -l 141.046 -i all --ibeos -j 4 || true
    fi
    cd 141.046_*
```

Backup code: relval_fix_check.sh (2/3)

scripts/relval_fix_check.sh, lines 35 to 67

```
    # of the 321 RAW files DAS lists for run 369978 (tape only), only this one is in the IB EOS cache
    echo "root://eoscms.cern.ch//eos/cms/store/user/cmsbuild/store/data/Run2023D/EGamma0/RAW/v1/000/369/978/00000/0068cd83-0af4-4efc-9155-b2521e558ebb.root" >
step1_dasquery.log
    grep -E "cmsDriver.py step[234] " cmdLog > steps234.sh
    cat steps234.sh
    bash steps234.sh
    ls -l step3_inNANOEDMAOD.root
    cd $WORK/run_clean
fi

echo "=== step 2: fix area with the cherry-picked commit"
cd $WORK
if [ ! -d fix ]; then
    scram project -n fix CMSSW $RELEASE
fi
cd fix/src
eval $(scramv1 runtime -sh)
if [ ! -d PhysicsTools/NanoAOD ]; then
    git cms-addpkg PhysicsTools/NanoAOD
    git fetch https://github.com/santanumahata/cmssw.git nano-photon-trigobj-unseeded
    git cherry-pick $FIX_COMMIT
fi
git diff $RELEASE --numstat
CHANGED=$(git diff $RELEASE --numstat | awk '{print $1, $2, $3}')
if [ "$CHANGED" != "1 1 $CFF" ]; then
    echo "ERROR: expected exactly one changed line in $CFF, got: $CHANGED"; exit 1
fi
git diff $RELEASE
scram b -j 8

echo "=== step 3: rerun steps 3 and 4 in the fix area on the clean step-2 output"
for CLEAN_DIR in $WORK/run_clean/12452.0_* $WORK/run_clean/141.046_*; do
    FIX_DIR=$WORK/run_fix/$(basename $CLEAN_DIR)
    mkdir -p $FIX_DIR && cd $FIX_DIR
```

Backup code: relval_fix_check.sh (3/3)

scripts/relval_fix_check.sh, lines 68 to 99

```
    ln -sf $CLEAN_DIR/step2.root step2.root
    grep -E "cmsDriver.py step[34] " $CLEAN_DIR/cmdLog > steps34.sh
    cat steps34.sh
    if [ ! -s steps34.sh ]; then
        echo "ERROR: no step3/step4 command in $CLEAN_DIR/cmdLog"; exit 1
    fi
    bash steps34.sh
done

echo "=== step 4: flat NANO from the EDM NANO of both workflows (both areas)"
for AREA in clean fix; do
    cd $WORK/$AREA/src && eval $(scramv1 runtime -sh)
    cd $WORK/run_$AREA/12452.0_*
    cmsDriver.py flat --step NONE --mc --conditions auto:phase1_2023_realistic --era Run3_2023 \
        --filein file:step3_inNANOEDMAODSIM.root --fileout file:step3_flatNANOAODSIM.root \
        --eventcontent NANOAODSIM --datatier NANOAODSIM -n -1 > flat_nano.log 2>&1
    cd $WORK/run_$AREA/141.046_*
    cmsDriver.py flat --step NONE --data --conditions auto:run3_data_relval --era Run3_2023 \
        --filein file:step3_inNANOEDMAOD.root --fileout file:step3_flatNANOAOD.root \
        --eventcontent NANOAOD --datatier NANOAOD -n -1 > flat_nano.log 2>&1
done

echo "=== step 5: log check"
for AREA in clean fix; do
    for LOG in $WORK/run_$AREA/*/step3_*.log; do
        echo "$LOG: exceptions $(grep -c "Exception" $LOG || true)," \
             "triggerObjectTable lines $(grep -c "triggerObjectTable" $LOG || true)"
        grep "triggerObjectTable" $LOG | head -5 || true
    done
done
echo "=== done"
```

Backup code: compare_dqm.py (1/2)

scripts/compare_dqm.py, lines 1 to 34

```
#!/usr/bin/env python3
"""List the histograms that differ between two harvested DQM files.

Usage: python3 compare_dqm.py <clean DQM_V0001_*.root> <fix DQM_V0001_*.root> <out.json>
"""
import json
import sys

import numpy as np
import uproot


def histograms(path):
    # Total: time O(n_hist * n_bins), space O(n_hist * n_bins)
    found = {}
    with uproot.open(path) as root_file:
        for name, classname in root_file.classnames().items():  # dict scan O(n)   | every stored object
            if not classname.startswith(("TH1", "TH2", "TH3", "TProfile")):
                continue
            found[name.split(";")[0]] = root_file[name].values(flow=True)  # array O(bins) | bin contents
    return found


def main():
    # Total: time O(n_hist * n_bins), space O(n_hist * n_bins)
    clean_path, fix_path, out_path = sys.argv[1:4]
    clean, fix = histograms(clean_path), histograms(fix_path)
    only_clean = sorted(set(clean) - set(fix))                # set diff O(n)    | histograms lost by the fix
    only_fix = sorted(set(fix) - set(clean))                  # set diff O(n)    | histograms added by the fix
    differing = sorted(name for name in set(clean) & set(fix)  # set join O(n)    | same name, different bins
                       if clean[name].shape != fix[name].shape or not np.array_equal(clean[name], fix[name]))
    result = {
        "clean": clean_path,
        "fix": fix_path,
```

Backup code: compare_dqm.py (2/2)

scripts/compare_dqm.py, lines 35 to 48

```
        "n_histograms_clean": len(clean),
        "n_histograms_fix": len(fix),
        "only_in_clean": only_clean,
        "only_in_fix": only_fix,
        "differing": differing,
    }
    with open(out_path, "w") as out:
        json.dump(result, out, indent=1)
    print(json.dumps({k: v for k, v in result.items()}, indent=1)[:6000])


if __name__ == "__main__":
    main()
```

Backup code: commands as run: (1/13)

slides/code/commands_as_run.sh, lines 1 to 32

```
# Shell commands I ran by hand for the 5 November 2026 checks, copied verbatim from the session log.
# They ran from my laptop; the work happened on lxplus over ssh. Not a script to execute as a whole.

# ===== 1. DAS lookups

# --- Query DAS for NanoAOD datasets per version pattern
ssh -o BatchMode=yes lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  for p in \
    "/EGamma/Run2018*-UL2018_MiniAODv2_NanoAODv9*/NANOAOD" \
    "/EGamma/Run2022*-PromptNanoAODv10*/NANOAOD" \
    "/EGamma/Run2022*-PromptNanoAODv11*/NANOAOD" \
    "/EGamma/Run2022*-22Sep2023-v*/NANOAOD" \
    "/GluGluHtoGG*/Run3Summer22*NanoAODv12*/NANOAODSIM" \
    "/EGamma/Run2022*-16Dec2023-v*/NANOAOD" \
    "/GluGluHto*G*/Run3Summer23NanoAODv13*/NANOAODSIM" \
    "/EGamma*/Run2024*-PromptReco-v*/NANOAOD" \
    "/GluGluHto*G*/*NanoAODv14*/NANOAODSIM" \
    "/GluGluHto*G*/Run3Summer2*NanoAODv15*/NANOAODSIM" \
    "/EGamma*/Run202*NanoAODv15*/NANOAOD"
  do echo "== $p"; dasgoclient -query="dataset=$p" 2>&1 | head -8; done' 2>&1 | grep -v '^\*'

# --- Query DAS with wider v14/v15 patterns
ssh -o BatchMode=yes lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  for p in \
    "/GluGluH*G*/*NanoAODv14*/NANOAODSIM" \
    "/GluGluH*G*/Run3Summer24*/NANOAODSIM" \
    "/GluGluH*G*/*NanoAODv15*/NANOAODSIM" \
    "/EGamma*/Run2024*NanoAODv15*/NANOAOD"
  do echo "== $p"; dasgoclient -query="dataset=$p" 2>&1 | head -8; done' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 1. DAS lookups (2/13)

slides/code/commands_as_run.sh, lines 33 to 60

```
# --- Get first file LFN for each chosen dataset
ssh -o BatchMode=yes lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  for d in \
    /EGamma/Run2018D-UL2018_MiniAODv2_NanoAODv9-v3/NANOAOD \
    /EGamma/Run2022D-PromptNanoAODv10_v2-v1/NANOAOD \
    /EGamma/Run2022F-PromptNanoAODv11_v1-v2/NANOAOD \
    /GluGluHtoGG_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer22NanoAODv12-130X_mcRun3_2022_realistic_v5-v2/NANOAODSIM \
    /EGamma/Run2022D-22Sep2023-v1/NANOAOD \
    /GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23NanoAODv13-133X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/NANOAODSIM \
    /EGamma/Run2022D-16Dec2023-v1/NANOAOD \
    /EGamma0/Run2024D-PromptReco-v1/NANOAOD \
    /GluGluH-Hto2G_Par-M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/RunIII2024Summer24NanoAODv15-150X_mcRun3_2024_realistic_v2-v2/NANOAODSIM \
    /EGamma/Run2022D-NanoAODv15-v1/NANOAOD
  do echo "== $d"; dasgoclient -query="file dataset=$d" 2>&1 | head -1; done' 2>&1 | grep -v '^\*'

# --- Look up DAS datasets needed for tasks A, B, C
ssh -o BatchMode=yes lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  for p in \
    "/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23MiniAODv4-130X_mcRun3_2023_realistic_v14-v2/MINIAODSIM" \
    "/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23MiniAODv4*/MINIAODSIM" \
    "/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23NanoAODv1*/NANOAODSIM" \
    "/GluGluHto*G*/Run3Summer23NanoAODv15*/NANOAODSIM" \
    "/EGamma0/Run2024*-MINIv6NANOv15-v*/NANOAOD" \
    "/GluGluHtoGG*/*Winter24*NanoAOD*/NANOAODSIM"
  do echo "== $p"; dasgoclient -query="dataset=$p" 2>&1 | head -6; done' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 1. DAS lookups (3/13)

slides/code/commands_as_run.sh, lines 61 to 81

```
# --- Check MiniAOD dataset status, central-file parent, wider patterns
ssh -o BatchMode=yes lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  for d in /GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23MiniAODv4-130X_mcRun3_2023_realistic_v14-v2/MINIAODSIM /GluGluHto2G_M-
125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23MiniAODv4-130X_mcRun3_2023_realistic_v15_ext1-v4/MINIAODSIM; do
    echo "== $d"; dasgoclient -query="dataset dataset=$d status=*" -json 2>/dev/null | grep -o "\"dataset_access_type\":\"[A-Z]*\"" | head -1; dasgoclient -query="summary
dataset=$d" 2>&1 | head -2; done
  echo "== parent of central v13 file"
  dasgoclient -query="parent file=/store/mc/Run3Summer23NanoAODv13/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-
pythia8/NANOAODSIM/133X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/2820000/4f45fe87-a958-4c8f-b88f-c322fec857f3.root"
  for p in "/*/Run3Summer23NanoAODv15*/NANOAODSIM" "/GluGluH*/*Winter24*/NANOAODSIM" "/*/RunIII2024Winter24NanoAOD*/NANOAODSIM"; do echo "== $p"; dasgoclient -
query="dataset=$p" 2>&1 | head -4; done' 2>&1 | grep -v '^\*'

# --- Get first file of 2024D MINIv6NANOv15 and Winter24 datasets
ssh -o BatchMode=yes -o ConnectTimeout=30 lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  for d in /EGamma0/Run2024D-MINIv6NANOv15-v1/NANOAOD /GluGluHToGG_M-125_TuneCP5_13p6TeV_powheg-pythia8/Run3Winter24NanoAOD-133X_mcRun3_2024_realistic_v9-v2/NANOAODSIM; do
    echo "== $d"; dasgoclient -query="file dataset=$d" | head -1; done' 2>&1 | grep -v '^\*'

# --- Check DAS sites and IB EOS cache for the RAW file
timeout 120 ssh -o BatchMode=yes -o ConnectTimeout=30 lxplus.cern.ch 'export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms;
F=/store/data/Run2023D/EGamma0/RAW/v1/000/369/978/00000/0068cd83-0af4-4efc-9155-b2521e558ebb.root; echo "== DAS sites"; dasgoclient -query="site file=$F"; echo "== ibeos
cache"; ls -la /eos/cms/store/user/cmsbuild$F 2>&1 | head -2; ls /eos/cms/store/user/cmsbuild/store/data/Run2023D/EGamma0/RAW/v1/000/369/978/00000/ 2>&1 | head -3' 2>&1 |
grep -v '^\*'

# --- Check which 141.046 input files exist in IB EOS cache
timeout 120 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ConnectTimeout=30 smahata@lxplus967.cern.ch 'cd
/eos/user/s/smahata/trigobj_check/relval_16_1_3/run_clean/141.046_*; wc -l < step1_dasquery.log; cat step1_lumiRanges.log | head -c 300; echo; for f in $(cat
step1_dasquery.log); do if [ -f /eos/cms/store/user/cmsbuild$f ]; then echo "cached $f"; else echo "missing $f"; fi; done | sort | uniq -c -w 7 | head; ls
/eos/cms/store/user/cmsbuild/store/data/Run2023D/EGamma0/RAW/v1/000/369/978/00000/ | wc -l' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 2. Measuring the NanoAOD files (4/13)

slides/code/commands_as_run.sh, lines 82 to 105

```
# ===== 2. Measuring the NanoAOD files

# --- Copy script to lxplus and run v13 smoke test
cd <session folder> && scp -q -o BatchMode=yes measure_trigobj.py lxplus.cern.ch:/eos/user/s/smahata/trigobj_check/ 2>&1 | grep -v '^\*'; ssh -o BatchMode=yes lxplus.cern.ch
'
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  cd /eos/user/s/smahata/trigobj_check
  mkdir -p nanoaod-trigobj-notes/results
  python3 measure_trigobj.py v13 \
    /GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23NanoAODv13-133X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/NANOAODSIM \
    root://cms-xrd-global.cern.ch//store/mc/Run3Summer23NanoAODv13/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-
pythia8/NANOAODSIM/133X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/2820000/4f45fe87-a958-4c8f-b88f-c322fec857f3.root \
    nanoaod-trigobj-notes/results/v13_GluGluHto2G_M125_Run3Summer23.json' 2>&1 | grep -v '^\*'

# --- Run measurement on the remaining 9 NanoAOD files
ssh -o BatchMode=yes lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  cd /eos/user/s/smahata/trigobj_check
  R=root://cms-xrd-global.cern.ch/
  run() { echo "##### $1 $4"; python3 measure_trigobj.py "$1" "$2" "$R$3" "nanoaod-trigobj-notes/results/$4.json" 2>&1 | grep -E "WARNING|Error|error|\"
(events_read|filterBits_interpretation|n_photon_bits_documented|fired_Mass90|fired_DoubleEle33_CaloIdL_MW|electron_CaloIdL_MW_unseeded_bit|electrons_with_CaloIdL_MW_unseeded
_bit|n_id2_MET|n_id1515_BoostedTau|n_id22_Photon|events_with_unseeded_photon_bit)\""; }
  run v9  /EGamma/Run2018D-UL2018_MiniAODv2_NanoAODv9-v3/NANOAOD /store/data/Run2018D/EGamma/NANOAOD/UL2018_MiniAODv2_NanoAODv9-v3/250000/566E12C7-CD3E-3F4D-A657-
A71FF4CB2637.root v9_EGamma_Run2018D_UL2018NanoAODv9
  run v10 /EGamma/Run2022D-PromptNanoAODv10_v2-v1/NANOAOD /store/data/Run2022D/EGamma/NANOAOD/PromptNanoAODv10_v2-v1/2520000/3f1fe826-1729-4ba9-83b9-d28a2cfd2b88.root
v10_EGamma_Run2022D_PromptNanoAODv10
  run v11 /EGamma/Run2022F-PromptNanoAODv11_v1-v2/NANOAOD /store/data/Run2022F/EGamma/NANOAOD/PromptNanoAODv11_v1-v2/80000/6d07f113-b587-45b3-8db6-c2928358ae0b.root
v11_EGamma_Run2022F_PromptNanoAODv11
  run v12 /GluGluHtoGG_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer22NanoAODv12-130X_mcRun3_2022_realistic_v5-v2/NANOAODSIM
/store/mc/Run3Summer22NanoAODv12/GluGluHtoGG_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/NANOAODSIM/130X_mcRun3_2022_realistic_v5-v2/2530000/fd1a30c4-ac59-4c33-8245-
719b82fad9f0.root v12_GluGluHtoGG_M125_Run3Summer22
```

Backup code: commands as run: 2. Measuring the NanoAOD files (5/13)

slides/code/commands_as_run.sh, lines 106 to 123

```
  run v12 /EGamma/Run2022D-22Sep2023-v1/NANOAOD /store/data/Run2022D/EGamma/NANOAOD/22Sep2023-v1/2530000/e5762711-8726-4523-8891-32efa9ddb381.root
v12_EGamma_Run2022D_22Sep2023
  run v13 /EGamma/Run2022D-16Dec2023-v1/NANOAOD /store/data/Run2022D/EGamma/NANOAOD/16Dec2023-v1/2550000/499b6a69-bd7e-4bce-bd6d-4ab4f250ea2d.root
v13_EGamma_Run2022D_16Dec2023
  run v14 /EGamma0/Run2024D-PromptReco-v1/NANOAOD /store/data/Run2024D/EGamma0/NANOAOD/PromptReco-v1/000/380/306/00000/9bfae790-c112-48bf-bc54-ef1feaa74a96.root
v14_EGamma0_Run2024D_PromptReco
  run v15 /GluGluH-Hto2G_Par-M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/RunIII2024Summer24NanoAODv15-150X_mcRun3_2024_realistic_v2-v2/NANOAODSIM
/store/mc/RunIII2024Summer24NanoAODv15/GluGluH-Hto2G_Par-M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/NANOAODSIM/150X_mcRun3_2024_realistic_v2-v2/120000/acebfb52-a25b-48bc-
b9f6-80fe54a98d56.root v15_GluGluHHto2G_M125_RunIII2024Summer24
  run v15 /EGamma/Run2022D-NanoAODv15-v1/NANOAOD /store/data/Run2022D/EGamma/NANOAOD/NanoAODv15-v1/2820000/41628bc7-c70f-4bf7-b2a4-e755aea819fd.root
v15_EGamma_Run2022D_NanoAODv15
  echo ALLDONE' 2>&1 | grep -v '^\*'

# --- Measure 2024D MINIv6NANOv15 and Winter24 files
ssh -o BatchMode=yes -o ServerAliveInterval=15 lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  cd /eos/user/s/smahata/trigobj_check
  R=root://cms-xrd-global.cern.ch/

K="events_read|events_in_file|filterBits_interpretation|n_photon_bits_documented|fired_Mass90|photons_with_bit|R9|TrackIso|EG18|seeded|hltHtEcal|electron_CaloIdL|electrons_w
ith|n_id2|n_id1515|n_id22|events_with_unseeded|nTrigObj_mean|file_size|WARNING|NOTE|Error"
  python3 measure_trigobj.py v15 /EGamma0/Run2024D-MINIv6NANOv15-v1/NANOAOD $R/store/data/Run2024D/EGamma0/NANOAOD/MINIv6NANOv15-v1/110000/9d4e97c7-f23a-4cff-ac72-
58488e544076.root nanoaod-trigobj-notes/results/v15_EGamma0_Run2024D_MINIv6NANOv15.json 2>&1 | grep -E "$K" | head -40
  echo "#####"
  python3 measure_trigobj.py Winter24_133X /GluGluHToGG_M-125_TuneCP5_13p6TeV_powheg-pythia8/Run3Winter24NanoAOD-133X_mcRun3_2024_realistic_v9-v2/NANOAODSIM
$R/store/mc/Run3Winter24NanoAOD/GluGluHToGG_M-125_TuneCP5_13p6TeV_powheg-pythia8/NANOAODSIM/133X_mcRun3_2024_realistic_v9-v2/80000/68618af9-2d38-4ef7-9aa9-2d0c30a6e3f1.root
nanoaod-trigobj-notes/results/Winter24_133X_GluGluHToGG_M125_powheg.json 2>&1 | grep -E "$K" | head -40' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 3. Fix test, v13 and v15 (6/13)

slides/code/commands_as_run.sh, lines 124 to 149

```
# ===== 3. Fix test, v13 and v15

# --- Copy scripts and start NANO runs on lxplus967 as smahata
cd <session folder> && O="-o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=15"; scp -q $O make_fixed_nano.sh measure_trigobj.py
smahata@lxplus967.cern.ch:/eos/user/s/smahata/trigobj_check/ 2>&1 | grep -v '^\*'; ssh $O smahata@lxplus967.cern.ch 'cd /eos/user/s/smahata/trigobj_check && nohup bash -c
"bash make_fixed_nano.sh v13; bash make_fixed_nano.sh v15" > make_fixed_nano.log 2>&1 < /dev/null & sleep 8; hostname; head -5
/eos/user/s/smahata/trigobj_check/make_fixed_nano.log' 2>&1 | grep -v '^\*'

# --- Rerun v13 nano_fix cmsRun on lxplus967 in held-open session
ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  cd /eos/user/s/smahata/trigobj_check/cmssw/CMSSW_13_3_1_patch1/src && eval $(scramv1 runtime -sh)
  git diff --numstat
  cd /eos/user/s/smahata/trigobj_check/cmssw/v13
  echo "≡≡≡ v13 nano_fix rerun start $(date)"
  cmsRun nano_fix_cfg.py > nano_fix_rerun.log 2>&1
  echo "≡≡≡ v13 nano_fix rerun exit $? $(date)"
  ls -l nano_nofix.root nano_fix.root' 2>&1 | grep -v '^\*'

# --- Run v15 fix-test NANO pair on another lxplus node
ssh -o BatchMode=yes -o ServerAliveInterval=30 lxplus.cern.ch 'hostname; cd /eos/user/s/smahata/trigobj_check && bash make_fixed_nano.sh v15 > make_fixed_nano_v15.log 2>&1;
echo "v15 exit $? $(date)"; grep -E "^≡≡≡ |^ERROR:|^[0-9]+: +sel|^[-+] +sel" make_fixed_nano_v15.log' 2>&1 | grep -v '^\*'

# --- Measure central v13 file into fix_test results
ssh -o BatchMode=yes -o ServerAliveInterval=30 lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  cd /eos/user/s/smahata/trigobj_check && mkdir -p nanoaod-trigobj-notes/results/fix_test
  python3 measure_trigobj.py v13_central /GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-pythia8/Run3Summer23NanoAODv13-133X_mcRun3_2023_realistic_ForNanov13_v1_ext1-
v4/NANOAODSIM root://cms-xrd-global.cern.ch//store/mc/Run3Summer23NanoAODv13/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-
pythia8/NANOAODSIM/133X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/2820000/4f45fe87-a958-4c8f-b88f-c322fec857f3.root nanoaod-trigobj-notes/results/fix_test/central_v13.json
2>&1 | grep -E "events_read|fired_Mass90|nTrigObj_mean|file_size|events_with_unseeded|fraction|photons_per|Error|WARN"' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 3. Fix test, v13 and v15 (7/13)

slides/code/commands_as_run.sh, lines 150 to 174

```
# --- Measure v13 nano_nofix and nano_fix with references
ssh -o BatchMode=yes -o ServerAliveInterval=30 lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  cd /eos/user/s/smahata/trigobj_check; V=cmssw/v13; O=nanoaod-trigobj-notes/results/fix_test
  CENTRAL=root://cms-xrd-global.cern.ch//store/mc/Run3Summer23NanoAODv13/GluGluHto2G_M-125_TuneCP5_13p6TeV_amcatnloFXFX-
pythia8/NANOAODSIM/133X_mcRun3_2023_realistic_ForNanov13_v1_ext1-v4/2820000/4f45fe87-a958-4c8f-b88f-c322fec857f3.root

K="events_read|fired_Mass90|R9|TrackIso|EG18|seeded|hltHtEcal|nTrigObj_mean|file_size|events_with_unseeded|fraction|photons_per|n_id22|\"n\"|min|median|below|reference|event
s_in|identical|Error|WARN|NOTE"
  python3 measure_trigobj.py v13_nofix MiniAOD-parent-of-central-v13 $V/nano_nofix.root $O/nano_nofix.json $CENTRAL 2>&1 | grep -E "$K"
  echo "#####"
  python3 measure_trigobj.py v13_fix MiniAOD-parent-of-central-v13 $V/nano_fix.root $O/nano_fix.json $V/nano_nofix.root 2>&1 | grep -E "$K"
  python3 -c "import json; a=json.load(open(\"$O/nano_nofix.json\")); b=json.load(open(\"$O/nano_fix.json\")); print(\"objects_per_id nofix\", a[\"objects_per_id\"]);
print(\"objects_per_id fix  \", b[\"objects_per_id\"]); print(\"id≠22 bit counts identical:\", a[\"bit_counts_per_id_without_22\"]==b[\"bit_counts_per_id_without_22\"])"'
2>&1 | grep -v '^\*'

# --- Measure v15 nano_nofix and nano_fix
ssh -o BatchMode=yes -o ServerAliveInterval=30 lxplus.cern.ch '
  export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  cd /eos/user/s/smahata/trigobj_check; V=cmssw/v15; O=nanoaod-trigobj-notes/results/fix_test
  ls -l $V/*.root

K="events_read|events_in_file|interpretation|n_photon_bits|fired_Mass90|R9|TrackIso|EG18|seeded|hltHtEcal|nTrigObj_mean|file_size|events_with_unseeded|fraction|photons_per|n
_id22|\"n\"|min|median|below|events_in|identical|Error|WARN|NOTE"
  python3 measure_trigobj.py v15_nofix MiniAOD-parent-of-central-v13 $V/nano_nofix.root $O/v15_nano_nofix.json 2>&1 | grep -E "$K"
  echo "#####"
  python3 measure_trigobj.py v15_fix MiniAOD-parent-of-central-v13 $V/nano_fix.root $O/v15_nano_fix.json $V/nano_nofix.root 2>&1 | grep -E "$K"
  python3 -c "import json; a=json.load(open(\"$O/v15_nano_nofix.json\")); b=json.load(open(\"$O/v15_nano_fix.json\")); print(\"objects_per_id nofix\",
a[\"objects_per_id\"]); print(\"objects_per_id fix  \", b[\"objects_per_id\"]); print(\"id≠22 bit counts identical:\",
a[\"bit_counts_per_id_without_22\"]==b[\"bit_counts_per_id_without_22\"])"' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 4. Relval in CMSSW_16_1_3 (8/13)

slides/code/commands_as_run.sh, lines 175 to 199

```
# ===== 4. Relval in CMSSW_16_1_3

# --- List the five relval workflows in CMSSW_16_1_3
ssh -o BatchMode=yes -o ConnectTimeout=30 -o ServerAliveInterval=15 lxplus.cern.ch 'source /cvmfs/cms.cern.ch/cmsset_default.sh
  cd /cvmfs/cms.cern.ch/el9_amd64_gcc13/cms/cmssw/CMSSW_16_1_3/src && eval $(scramv1 runtime -sh)
  mkdir -p /tmp/smahata_rtm && cd /tmp/smahata_rtm && runTheMatrix.py -n -e -l 12452.0,12852.0,12434.0,12834.0,141.046 2>&1 | grep -v "^$" | cut -c1-400 | head -60' 2>&1 |
grep -v '^\*'

# --- List relval workflows with upgrade and default matrices
ssh -o BatchMode=yes -o ConnectTimeout=30 -o ServerAliveInterval=15 lxplus.cern.ch 'source /cvmfs/cms.cern.ch/cmsset_default.sh
  cd /cvmfs/cms.cern.ch/el9_amd64_gcc13/cms/cmssw/CMSSW_16_1_3/src && eval $(scramv1 runtime -sh)
  mkdir -p /tmp/smahata_rtm && cd /tmp/smahata_rtm
  echo "##### upgrade 12452/12852"; runTheMatrix.py -n -e -w upgrade -l 12452.0,12852.0 2>&1 | grep -v "^processing\|^ignoring\|^$" | cut -c1-300 | head -30
  echo "##### default 12434/12834/141.046"; runTheMatrix.py -n -e -l 12434.0,12834.0,141.046 2>&1 | grep -v "^processing\|^ignoring\|^$" | cut -c1-300 | head -40' 2>&1 |
grep -v '^\*'

# --- Run relval check with cached RAW file for 141.046
ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=30 smahata@lxplus967.cern.ch 'cd /eos/user/s/smahata/trigobj_check && nohup bash
relval_fix_check.sh > relval_fix_check.log 2>&1 < /dev/null; echo "relval exit code $?"; grep -E "^═══ |^ERROR:|tests passed|Step[0-9]┤^[0-9]+\s+[0-
9]+\s+Phys|exceptions|Fatal" relval_fix_check.log | cut -c1-250' 2>&1 | grep -v '^\*'

# --- Dry-run 141.046 to get full step3/step4 commands
timeout 300 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ConnectTimeout=30 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh; export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  cd /eos/user/s/smahata/trigobj_check/relval_16_1_3/clean/src && eval $(scramv1 runtime -sh)
  mkdir -p /tmp/smahata_dry && cd /tmp/smahata_dry && rm -rf 141.046_*
  runTheMatrix.py -l 141.046 -i all --ibeos --dryRun > dry.log 2>&1
  grep -E "cmsDriver.py step[34] " 141.046_*/cmdLog' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 4. Relval in CMSSW_16_1_3 (9/13)

slides/code/commands_as_run.sh, lines 200 to 233

```
# --- Run clean 141.046 steps 3-4 then the rest of relval script
ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh; export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  cd /eos/user/s/smahata/trigobj_check/relval_16_1_3/clean/src && eval $(scramv1 runtime -sh)
  cd /eos/user/s/smahata/trigobj_check/relval_16_1_3/run_clean/141.046_*
  grep -E "cmsDriver.py step[34] " /tmp/smahata_dry/141.046_*/cmdLog > steps34_from_dryrun.sh
  echo "# steps 3-4 below taken from runTheMatrix --dryRun (aborted run had not written them)" >> cmdLog
  cat steps34_from_dryrun.sh >> cmdLog
  echo "=== clean 141.046 steps 3-4 start $(date)"
  bash steps34_from_dryrun.sh; echo "steps34 exit $?"
  ls -l step3_inNANOEDMAOD.root step3_inDQM.root DQM_V0001*.root
  cd /eos/user/s/smahata/trigobj_check
  bash relval_fix_check.sh > relval_fix_check.log 2>&1; echo "relval exit code $? $(date)"
  grep -E "^=== |^ERROR:|^[0-9]+\s+[0-9]+\s+Phys|exceptions" relval_fix_check.log | cut -c1-250' 2>&1 | grep -v '^\*'

# --- Rerun 141.046 steps 3-4 in the fix area
ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh; export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  W=/eos/user/s/smahata/trigobj_check/relval_16_1_3
  cd $W/fix/src && eval $(scramv1 runtime -sh) && git diff CMSSW_16_1_3 --numstat
  cd $W/run_fix/141.046_RunEGamma2023D
  cp $W/run_clean/141.046_RunEGamma2023D/steps34_from_dryrun.sh .
  echo "=== fix 141.046 steps 3-4 start $(date)"
  bash steps34_from_dryrun.sh; echo "=== fix 141.046 steps 3-4 exit $? $(date)"
  ls -l step3_inNANOEDMAOD.root DQM_V0001*.root' 2>&1 | grep -v '^\*'

# --- Convert, measure and compare 12452.0 clean vs fix
timeout 900 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ConnectTimeout=60 -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh; export X509_USER_PROXY=/afs/cern.ch/user/s/smahata/private/x509up_cms
  W=/eos/user/s/smahata/trigobj_check/relval_16_1_3; T=/eos/user/s/smahata/trigobj_check
  for AREA in clean fix; do
    (cd $W/$AREA/src && eval $(scramv1 runtime -sh) && cd $W/run_$AREA/12452.0_* && \
     cmsDriver.py flat --step NONE --mc --conditions auto:phase1_2023_realistic --era Run3_2023 \
       --filein file:step3_inNANOEDMAODSIM.root --fileout file:step3_flatNANOAODSIM.root \
```

Backup code: commands as run: 4. Relval in CMSSW_16_1_3 (10/13)

slides/code/commands_as_run.sh, lines 234 to 255

```
      --eventcontent NANOAODSIM --datatier NANOAODSIM -n -1 > flat_nano.log 2>&1; echo "$AREA flat exit $?"; ls -l step3_flatNANOAODSIM.root)
  done
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  O=$T/nanoaod-trigobj-notes/results/relval_16_1_3; mkdir -p $O
  C=$(ls $W/run_clean/12452.0_*/step3_flatNANOAODSIM.root); F=$(ls $W/run_fix/12452.0_*/step3_flatNANOAODSIM.root)
  cd $T
  python3 measure_trigobj.py 16_1_3_clean wf12452.0 $C $O/wf12452_clean.json 2>&1 | grep -E
"events_read|fired_Mass90|n_photon_bits|n_id22|events_with_unseeded|fraction|nTrigObj_mean|Error"
  python3 measure_trigobj.py 16_1_3_fix wf12452.0 $F $O/wf12452_fix.json $C 2>&1 | grep -E
"events_read|fired_Mass90|R9Unseeded|TrackIsoUnseeded|n_id22|events_with_unseeded|fraction|nTrigObj_mean|min|identical|Error"
  python3 -c "import json; a=json.load(open(\"$O/wf12452_clean.json\")); b=json.load(open(\"$O/wf12452_fix.json\")); print(\"ids clean\", a[\"objects_per_id\"]); print(\"ids
fix  \", b[\"objects_per_id\"]); print(\"id≠22 bits identical:\", a[\"bit_counts_per_id_without_22\"]==b[\"bit_counts_per_id_without_22\"]); print(\"photon bits clean\",
a[\"photons_with_bit\"]); print(\"photon bits fix  \", b[\"photons_with_bit\"])"
  python3 compare_dqm.py $(ls $W/run_clean/12452.0_*/DQM_V0001*.root) $(ls $W/run_fix/12452.0_*/DQM_V0001*.root) $O/wf12452_dqm_diff.json 2>&1 | head -60
  echo "--- logs"
  for A in clean fix; do L=$(ls $W/run_$A/12452.0_*/step3_*.log); echo "$A step3: Exception lines $(grep -c Exception $L), triggerObjectTable lines $(grep -c
triggerObjectTable $L)"; grep triggerObjectTable $L | head -3; done
  L=$(ls $W/run_clean/141.046_*/step3_*.log); echo "clean 141.046 step3: Exception lines $(grep -c Exception $L), triggerObjectTable lines $(grep -c triggerObjectTable $L)"'
2>&1 | grep -v '^\*' | cut -c1-300

# --- Rerun 12452.0 DQM comparison without time limit
ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh
  W=/eos/user/s/smahata/trigobj_check/relval_16_1_3; T=/eos/user/s/smahata/trigobj_check
  cd $T && echo "start $(date)"
  python3 compare_dqm.py $(ls $W/run_clean/12452.0_*/DQM_V0001*.root) $(ls $W/run_fix/12452.0_*/DQM_V0001*.root) nanoaod-trigobj-
notes/results/relval_16_1_3/wf12452_dqm_diff.json > /dev/null 2>&1; echo "exit $? $(date)"
  python3 -c "import json; d=json.load(open(\"nanoaod-trigobj-notes/results/relval_16_1_3/wf12452_dqm_diff.json\")); print(d[\"n_histograms_clean\"],
d[\"n_histograms_fix\"], len(d[\"only_in_clean\"]), len(d[\"only_in_fix\"]), len(d[\"differing\"])); print(\"\n\".join(d[\"differing\"][:40]))"' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 4. Relval in CMSSW_16_1_3 (11/13)

slides/code/commands_as_run.sh, lines 256 to 279

```
# --- Convert and measure clean 141.046 NANO
timeout 1200 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ConnectTimeout=60 -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh; W=/eos/user/s/smahata/trigobj_check/relval_16_1_3; T=/eos/user/s/smahata/trigobj_check
  (cd $W/clean/src && eval $(scramv1 runtime -sh) && cd $W/run_clean/141.046_RunEGamma2023D && \
   cmsDriver.py flat --step NONE --data --conditions auto:run3_data_relval --era Run3_2023 \
     --filein file:step3_inNANOEDMAOD.root --fileout file:step3_flatNANOAOD.root \
     --eventcontent NANOAOD --datatier NANOAOD -n -1 > flat_nano.log 2>&1; echo "flat exit $?"; ls -l step3_flatNANOAOD.root)
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh; cd $T
  python3 measure_trigobj.py 16_1_3_clean wf141.046 $W/run_clean/141.046_RunEGamma2023D/step3_flatNANOAOD.root nanoaod-trigobj-
notes/results/relval_16_1_3/wf141046_clean.json 2>&1 | grep -E
"events_read|n_photon_bits|fired_Mass90|R9Unseeded|TrackIsoUnseeded|seeded_|n_id22|events_with_unseeded|nTrigObj_mean|Error"' 2>&1 | grep -v '^\*'

# --- Convert, measure, check logs and DQM for 141.046 fix
ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ServerAliveInterval=30 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh; W=/eos/user/s/smahata/trigobj_check/relval_16_1_3; T=/eos/user/s/smahata/trigobj_check; O=$T/nanoaod-trigobj-
notes/results/relval_16_1_3
  (cd $W/fix/src && eval $(scramv1 runtime -sh) && cd $W/run_fix/141.046_RunEGamma2023D && \
   cmsDriver.py flat --step NONE --data --conditions auto:run3_data_relval --era Run3_2023 \
     --filein file:step3_inNANOEDMAOD.root --fileout file:step3_flatNANOAOD.root \
     --eventcontent NANOAOD --datatier NANOAOD -n -1 > flat_nano.log 2>&1; echo "flat exit $?")
  L=$W/run_fix/141.046_RunEGamma2023D/step3_RunEGamma2023D.log; echo "fix 141.046 step3: Exception $(grep -c Exception $L), triggerObjectTable $(grep -c triggerObjectTable
$L), events $(grep -c "Begin processing" $L)"
  L=$W/run_clean/141.046_RunEGamma2023D/step3_RunEGamma2023D.log; echo "clean 141.046 step3 events $(grep -c "Begin processing" $L)"
  source /cvmfs/sft.cern.ch/lcg/views/LCG_107/x86_64-el9-gcc13-opt/setup.sh; cd $T
  python3 measure_trigobj.py 16_1_3_fix wf141.046 $W/run_fix/141.046_RunEGamma2023D/step3_flatNANOAOD.root $O/wf141046_fix.json
$W/run_clean/141.046_RunEGamma2023D/step3_flatNANOAOD.root 2>&1 | grep -E "events_read|fired_Mass90|n_id22|events_with_unseeded|nTrigObj_mean|events_in|identical|Error"
  python3 compare_dqm.py $W/run_clean/141.046_RunEGamma2023D/DQM_V0001*.root $W/run_fix/141.046_RunEGamma2023D/DQM_V0001*.root $O/wf141046_dqm_diff.json > /dev/null 2>&1;
echo "dqm exit $?"
  python3 -c "import json; d=json.load(open(\"$O/wf141046_dqm_diff.json\")); print(d[\"n_histograms_clean\"], d[\"n_histograms_fix\"], len(d[\"only_in_clean\"]),
len(d[\"only_in_fix\"]), len(d[\"differing\"])); print(\"\n\".join(x for x in d[\"differing\"] if \"TimerService\" not in x)[:3000])"' 2>&1 | grep -v '^\*'
```

Backup code: commands as run: 4. Relval in CMSSW_16_1_3 (12/13)

slides/code/commands_as_run.sh, lines 280 to 300

```
# --- Inspect trigger products in 141.046 MINIAOD and NANO
timeout 600 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ConnectTimeout=60 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh; W=/eos/user/s/smahata/trigobj_check/relval_16_1_3
  cd $W/clean/src && eval $(scramv1 runtime -sh) && cd $W/run_clean/141.046_RunEGamma2023D
  echo "== MINIAOD trigger products"; edmEventSize -v step3_inMINIAOD.root 2>/dev/null | grep -iE "trigger|TriggerResults" | head
  echo "== EDM NANO TrigObj"; edmEventSize -v step3_inNANOEDMAOD.root 2>/dev/null | grep -iE "trig" | head
  echo "== cfg"; grep -nE "unpackedPatTrigger|patTrigger\b|triggerResults|hltProcess|slimmedPatTrigger" step3_RAW2DIGI*.py 2>/dev/null | head; ls *.py' 2>&1 | grep -v '^\*'
| cut -c1-200

# --- Check HLT trigger summary products in relval outputs
timeout 600 ssh -o BatchMode=yes -o HostKeyAlias=lxplus.cern.ch -o ConnectTimeout=60 smahata@lxplus967.cern.ch '
  source /cvmfs/cms.cern.ch/cmsset_default.sh; W=/eos/user/s/smahata/trigobj_check/relval_16_1_3
  cd $W/clean/src && eval $(scramv1 runtime -sh) && cd $W/run_clean/141.046_RunEGamma2023D
  echo "== step2 (reHLT output) trigger summary products"; edmEventSize -v step2.root 2>/dev/null | grep -iE "TriggerEvent|hltTriggerSummary|TriggerResults" | head
  echo "== step3 AOD"; edmEventSize -v step3.root 2>/dev/null | grep -iE "TriggerEvent|hltTriggerSummary" | head
  cd $W/run_clean/12452.0_*; echo "== 12452 step2"; edmEventSize -v step2.root 2>/dev/null | grep -iE "TriggerEvent|hltTriggerSummary" | head; echo "== 12452 MINIAOD";
edmEventSize -v step3_inMINIAODSIM.root 2>/dev/null | grep -iE "slimmedPatTrigger" | head -3' 2>&1 | grep -v '^\*' | cut -c1-200

# ===== 5. HiggsDNA photon-bit usage

# --- Shallow-clone HiggsDNA repos and grep TrigObj usage
D=<scratch>/higgsdna_src; mkdir -p $D; cd $D; timeout 200 git clone -q --depth 1 https://gitlab.cern.ch/cms-analysis/general/HiggsDNA.git HiggsDNA 2>&1 | tail -2; timeout
200 git clone -q --depth 1 https://github.com/ZhenyuDongLL/HiggsDNA-bbggrun3.git bbgg 2>&1 | tail -2; git -C HiggsDNA log --oneline -1 | cat; git -C bbgg log --oneline -1 |
cat; grep -rn --include=*.py --include=*.json -E "trigger_match|TrigObj|filterBits" HiggsDNA | grep -v "^HiggsDNA/tests/data" | cut -c1-220 | head -60
```

Backup code: commands as run: 5. HiggsDNA photon-bit usage (13/13)

slides/code/commands_as_run.sh, lines 301 to 306

```
# --- Read HiggsDNA trigger-matching code and search bbgg fork
cd <scratch>/higgsdna_src; sed -n 230,262p HiggsDNA/higgs_dna/workflows/dystudies.py; echo ----; sed -n 300,325p HiggsDNA/higgs_dna/utils/misc_utils.py; echo ----; sed -n
630,645p HiggsDNA/higgs_dna/workflows/lowmass.py; echo ---- bbgg; grep -rn --include=*.py -E "trigger_match|TrigObj|filterBits" bbgg | cut -c1-200 | head -20; echo ----
other trig refs; grep -rln --include=*.py -iE "Diphoton30_22|hltEG22|Unseeded" HiggsDNA bbgg | head

# --- Show diphoton-trigger references in HiggsDNA systematics
cd <scratch>/higgsdna_src; grep -n -iE "Diphoton30_22|hltEG22|Unseeded" HiggsDNA/higgs_dna/systematics/event_weight_systematics.py HiggsDNA/higgs_dna/scripts/pull_files.py |
cut -c1-200 | head -12
```